The CIRIS Constitution: The Threat Model

The Threat Model, a volume of edition 1.0-rc15, released as guidance

Stewarded by Eric Moore

The Threat Model

What this is. What is trusted, why, and what happens when it is wrong: the adversary model of 1.13.3 with its conditioned cases, the transport, custody and privacy rules of Part 5 that answer it, and the trust assumptions a reviewer should attack first: the out-of-band anchor, the witness directory, the transparency logs where witnessed mode is off.

Whom it binds. Security reviewers and red teams by invitation; implementers by consequence, because every assumption named here is a rule somewhere in *The Wire* or *The Constitution*.

What it promises. That assumptions are named rather than implied; that each failure mode says what the record would show afterwards; and that every defence named here is a rule in *The Wire* or *The Constitution* whose evidence, or the absence of it, is a row in *The Evidence Register*.

What it leaves out. The ethics, which are *The Charter*; the composition policies that a consumer applies after the adversary has been accounted for, which are *The Reader's Policies*; and the concerns ledger, the interoperability profiles and the record of what was attacked, which are Part 8 in *The Evidence Register*.

Where the rest lives. The mechanised half of this volume is the set of formal models the register names: each states the property it checks and the configurations under which it fails.

Part 1 — Foundation

1.13.3 Adversary model & privacy non-goals (normative)

Fidelity demands the format not overclaim what it protects. CEG, the CIRIS Epistemic Grammar (CC 8.1.1), makes confidentiality and integrity claims; this section bounds them. **The word "privacy" in this spec means exactly two things and no more: (1) content-holding confidentiality and (2) cohort-scoped visibility.** It does **not** mean metadata privacy, communication-graph privacy, or unobservability. Implementers and operators MUST NOT represent CEG as providing the stronger properties.

1.13.3.1 Non-goals: what omission does NOT buy

The following are **explicitly out of scope** at the base CEG/RET layer, the grammar of Part 2 and the transport substrate of Part 5; treating them as provided is an error:

- **Relationship-existence privacy.** The *existence* of a self-collective / family / community and its membership-change events are observable: `family_id` / `community_id` ride the envelope, and admission / removal / consensus-protocol changes emit `hard_case:*` reserved-prefix events (CC 3.4.2–3.4.4) into the log. An observer learns that a group exists, roughly how big it is, and when its membership churns.
- **Communication-graph / metadata privacy.** DNS-free member resolution (CC 4.4.3.2.4.1) plus Reticulum announce / path-request expose *who is reachable where*; the federation directory + `transport_destination` bindings name endpoints. A passive network observer or an honest-but-curious member can reconstruct a substantial portion of the **who-talks-to-whom** graph. Cohort scope hides *content*, not *contact*.
- **Traffic-analysis resistance.** Encrypted streams still leak via side channels the wire format does not pad or cover: the CC 5.3.3.3 signed-tree-head (STH) cadence (default T=2 s), the churn-driven key-cascade volume/timing (CC 5.1), and per-chunk size/rate. An observer can infer stream existence, approximate group size, churn rate, activity bursts, and often media bitrate class — without decrypting a byte.
- **Unobservability against a *global passive adversary*.** Base CEG/RET does not, on its own, defeat an adversary who observes every network link. At **federation (public-commons) scope** the transport reveals path endpoints and traffic patterns, so full sender/receiver unobservability *at that scope* is a **separate, opt-in** mechanism (the Anonymous Tier of CC 5.4.6 — layered onion routing). This is the residual non-goal. It does **not** mean CEG provides no anonymity: at every cohort scope below federation, structural anonymity *to outsiders* is the default — see CC 1.13.3.4.
- **Post-compromise security (PCS) for streams.** The CC 4.5.12.1 Option-A choice is forward-only: a member removed at epoch e cannot read epoch $e+1$, but a *compromised current member's* key is not self-healed by a key-update the way MLS PCS provides.

1.13.3.2 What the structural-invisibility primitive (CC 5.2) buys

Keeping every holding claim inside its own cohort (CC 5.2) gives `cohort_scope: self | family` content what this specification calls **content-holding confidentiality**: a non-member cannot *discover that the bytes exist via the substrate* and cannot *fetch* them (no holder is advertised;

the bytes are delivered only to admitted members via the at-rest key cascade). End-to-end content confidentiality is additionally provided by the per-epoch data-encryption key (DEK, hybrid X25519+ML-KEM-768) and AES-256-GCM. That is the whole of what omission buys.

1.13.3.3 Adversary classes (and where each is / is not addressed)

Adversary	Addressed	NOT addressed
Passive network observer	content confidentiality (AEAD + DEK); equivocation (STH)	comm-graph, traffic analysis, group size/churn inference
Honest-but-curious member	— (members see in-scope content by design)	can enumerate co-members + reconstruct local comm-graph
Malicious member	cannot forge others' attestations; removal is forward-secret on every gated roster (an ungated public stream carries no confidentiality claim, CC 5.1)	can leak content they were entitled to; metadata as above
Compromised substrate node	cannot read the bodies of self, family or private-community content (sealed under a DEK it holds no wrap for); CEG-native replication carries signed provenance	reads every attestation row and every structural fact it holds (envelopes, scores, key ids, scopes, instants, the directory, holding claims); can observe traffic patterns it routes
Equivocating producer	**mitigated on accountable streams** — per-stream STH (CC 5.3.3.3) + consistency proofs (CC 5.3.1.1): cannot show different chunk-K to different viewers nor rewrite mid-stream; a best-effort stream has a producer-signed root only	a best-effort stream: its root is the producer's own signature, so the producer can show two viewers two histories there
Equivocating founder (trust-root lineage)	**mitigated once a lineage runs witnessed** — witnessed lineage heads (CC 3.2 T6, CC 5.3.1): two witnessed heads for one lineage are `hard_case: lineage_equivocation`, never a silent first-seen fork; exposure bounded by the charter's `witness_cadence`; a stale prefix cannot be attached to (CC 3.2 T4a)	a lineage that runs with witnessed mode off has a single-signer anchor for as long as it does; a founder set that colludes with every witness of its own lineage; an attached node's view between cadence ticks
Placeholder identity (a registered key with no live agent behind it)	**addressed by construction** — CEG keeps no registry whose entry constitutes agent-hood: a key is an identifier, and standing comes only from attestations others make about it under policies the consumer chose (CC 2.3.2, CC 4.4.3.8 Policy A); an `agent` or `node` key MUST have a bound human steward, a `user` identity it is an admitted `identity_occurrence` of, to hold non-infrastructure membership (CC 3.2 steward-binding gate); liveness is never a durable claim — entitlement is attested, reachability is an ephemeral node-local fact that is never an attestation and never logged (CC 5.3.3.4), so presence in any directory never reads as "live"	minting a key costs nothing; the CC defines no discovery-time liveness predicate beyond Edge reachability at query time, so discovery tooling that hides unreachable identities is client policy; a self-attested `identity_occurrence` is self-assertion until cross-attested (CC 3.1.1)

Adversary	Addressed	NOT addressed
Coordinated reviewer capture (Sybil feedback)	**mitigated** — influence is gated by trust, not by registration: the conformance-floor policy composes only attesters in the consumer's pinned trust set (CC 4.4.3.8 Policy A), so a fresh key has no weight until someone the consumer trusts confers it, and the richer policies (CC 4.4.3) root in admitted membership; detector dimensions aggregate by median (CC 4.4.2); on the aggregation leg, correlated sources count as one at full correlation (the correlated-source discount, CC 6.1.2.1.2); `ratchet:flag: coordinated_voting_cluster` names the pattern (CC 3.1.6)	the default aggregation for `signed` dimensions is a plain mean of `score` × `confidence` inside the trust set (CC 4.4.2) with no bounded-influence rule, so one captured trusted attester moves it; `evidence_refs` are required only where a dimension's rules say so, so an attestation with no verifiable interaction behind it is admissible at the grammar level and filtered only by consumer policy; `ratchet:flag: *` is advisory and carries no evidence row
Census stuffer (forged census returns move a relief, a default, a cap or a vote base)	**bounded** — a valid return adds exactly one count to one bucket per field (the one-hot proof, CC 6.1.9); per-node composites publish the median bucket, which moves at most one bucket below a fifth of returns forged; voice counts are owner-bound, and the dominance gate refuses a composite one owner dominates	an adversary holding a real share of rooted owners moves counts by that share (the admission cost, not a census property); a forger that returns plausible values is not detected — the census bounds influence, it does not detect lies
Census deanonymiser (small cells, sparse regions, differencing consecutive composites)	**mitigated** — ten owners per published cell, continent-sized regions, levels never deltas, values rounded to a grid, bucketed returns, and no aggregator sees a return at all	an observer who controls all but one owner in a cell infers the last one (nine colluding owners in that cell)
Compelled aggregator or directory (subpoena, coerced operator)	**mitigated** — an aggregator holds uniformly random shares and a public partial; a return's value is exposed only if both aggregators of a period are compelled or collude; a share is deleted once summed; the directory holds no census data	the aggregator sees which node sent it a share in the period (participation, which is everyone's)
Global passive observer on census timing	**mitigated** — one fixed-size return per period at an offset from `H(return_token ‖ period)`; on a shaped link (CC 5.4.3) it replaces a cover frame	on an unshaped link the observer sees that a node participates, which every eligible node does
Withholding adversary (suppresses returns to shrink a count or a vote base)	**bounded** — the base falls at most a tenth per period and never below its floor; a fall of the count by more than a third freezes the base until one period returns two thirds of the count before the fall	a sustained silence lowers the base to its floor over many periods, in plain view
Poisoned aggregator (misstates, drops or invents returns)	**mitigated** — two aggregators, each partial checked against the other's commitment and share count, buckets against `source_count`, a contributor's inclusion path against the commitment; only serve seats or community members aggregate, and they are accountable	two colluding aggregators can drop returns or invent tokens up to their expected share, bounded to the rounding grid by the directory's count of live nodes; the door checks only that each recipient is a seat; the per-seat load check is statistical, not enforced

Operator guidance: cohort-scoped confidentiality and anonymity-to-outsiders are on by **default** (CC 1.13.3.4). If a deployment *additionally* requires unobservability against a **global passive adversary** at federation scope (e.g., under a totalitarian-threat model), it MUST layer

the Anonymous Tier once the tier ships; until then the residual is stated at CC 1.13.3.1, and base CEG/RET is not sufficient for that strongest model. **Using** the Anonymous Tier is the publisher's opt-in; **relaying** it for others is the default: every node relays onion traffic unless it opts out, and a serve node cannot opt out, so that running a relay identifies no one and every operator can say only that it runs the standard substrate (CC 5.4.6). Its first scope is federation-scope publication. State both the default protection and its residual limit in any user-facing privacy representation.

1.13.3.4 Anonymity defaults to the smallest cohort scope (normative)

Anonymity defaults to the smallest cohort scope consistent with a publication's intent; **federation (public-commons) scope is the opt-in**, not anonymity. This is the protective default, chosen over opt-in anonymity: under opt-in, the non-savvy vulnerable — an abuse victim, a teenager in a hostile household — fail to obtain a protection that a motivated adversary obtains trivially, so opt-in weakens protection only for those least able to navigate it. CEG therefore makes cohort-scoped confidentiality and anonymity toward outsiders the *default* — anonymity here means outsiders cannot see a cohort's content or map its addresses to it; it never means an unattributed sender, since every link is attributed to a key and a joining device is unowned but never anonymous (CC 3.2 (e), CC 5.3.3.5 E1) — content born and living at `self` / `family` emits no holding claim outside its cohort (the structural-invisibility MUST of CC 5.2), and `community` content is sealed, its holding claims staying within the cohort (CC 5.2, CC 4.4.3.2.1), the one exception being authorized infrastructure communities and public rooms, whose bodies are plaintext and whose holding claims sit at federation scope (CC 4.4.3.2.1) — where, for `self`, **"outsider" means any node not owner-bound to my owner**: the `self` cohort is one owner's own devices, so its boundary is well-defined only because ownership is **single-valued** (CC 3.2 single-owner invariant) — and reserves opt-in only for the strongest threat model: full unobservability against a global passive adversary at federation scope (CC 1.13.3.1).

CEG declines client-side scanning — it would be the surveillance backdoor this framework refuses — and does **not** claim to detect harmful content inside private cohorts. What CEG adds is **fails-secure governance** (the named-moderator existence invariant, CC 4.5.4) and a **substrate-protective perceptual-hash tripwire at every scope-*widening* promotion event** — a promotion into the Commons, or a post into a `public` room (CC 3.2), creates the Contribution that is hash-matched at that seam — never on the device, never inside a private cohort. This recovers the spreading-correlated child-safety catches (the harm vector that creates new victims) without scanning content that stays within its cohort.

The opt-in boundary is drawn at GPA-unobservability because it is the one protection that is genuinely costly (onion-routing latency, mixing, dedicated cover) *and* is only reached by an operator already taking the deliberate step of publishing to the public commons — so the differential-uptake argument that makes cohort anonymity a *default* does not reach it.

Part 5 — Transport & Substrate

5.1 Epoch keying + cascade (normative)

The stream-epoch DEK seals content **O(1)**; the per-subscriber `key_grant` cascade distributes the 32-byte epoch key **O(N)/epoch** = CC 4.4.3.4.1 Policy-L cascade applied to a community roster against a *rotating* key.

PQC at rest — `wrap_algorithm: v2` MANDATORY (normative). The epoch-DEK `key_grant` cascade wraps a DEK only for a kept stream (below), and a DEK that is wrapped MUST be wrapped with **`wrap_algorithm: v2`** — the hybrid X25519 + ML-KEM-768 construction (FIPS 203) — never `v1` (X25519-only). The DEK protects content that may persist indefinitely; a classical-only KEM is a harvest-now-decrypt-later exposure even though the content AEAD (AES-256-GCM, CC 5.3.3.1) and the wrap *signature* (Ed25519 + ML-DSA-65) are already PQC-safe. The versioned `KEY_GRANT_V1_INFO` HKDF context rotates cleanly to the v2 context. A Consumer MUST reject a streaming epoch grant carrying `wrap_algorithm: v1`. **Full PQC envelope for streaming**: content = AES-256-GCM (symmetric, PQC-safe) · DEK wrap = X25519+ML-KEM-768 · authenticity = Ed25519+ML-DSA-65 · hashes = SHA-256 (PQC-safe) · in-transit = CC 5.3.3.5 E1 two-layer hybrid (below).

One construction, one wire identifier (normative). The hybrid X25519 + ML-KEM-768 + AES-256-GCM + HKDF-SHA256 construction has **exactly one** wire identifier across the federation: **`x25519_mlkem768_aes256_gcm_hkdf_sha256`** (the CC 3.3.2 vocab variant `X25519MlKem768Aes256GcmHkdfSha256`). Producer, substrate, and every consumer MUST serialize this exact byte string, and MUST accept only this byte string, wherever the construction is named — the streaming epoch-DEK wrap here, the CC 4.4.3.4.1 self/family at-rest wrap, and the media-DEK wrap are the **same** construction under the **same** identifier, **not** per-field vocabularies; a consumer MUST NOT treat them as distinct algorithms and MUST NOT define a field-local spelling. The hyphen-separated form `x25519-mlkem768-aes256-gcm-hkdf-sha256` — and every other case-, separator- or ordering-variant — is a **non-conformant alias**: it MUST be rejected as an unknown `wrap_algorithm` under the CC 3.3.2 closed-set fail-secure decode rule, and a consumer MUST NOT normalize case or separators before comparison (normalization silently re-opens the ambiguity this clause closes, and would make two distinct future vocab rows collide). The snake_case form is canonical because it is the form already pinned by the CC 3.3.2 vocabulary and its `v1` sibling `hpke_rfc9180_base_x25519_aes_gcm`.

Unlinkable presentation — required as a property, staged as a path (normative). Credential-presentation unlinkability is a **required property commitment**: it is the presentation-layer form of CC 1.13.3.4's differential-uptake rule (protection must be structural — the vulnerable cannot be asked to manage key hygiene), and an assurance rung presented across communities MUST NOT become a cross-service correlation handle. Adoption is **staged**: portability (OR-of-N) applies now; the unlinkable presentation format is a **reserved upgrade slot** with **ZK-wrapped unmodified ML-DSA as the designated candidate** (FIPS 204 untouched, no re-issuance — proofs are over signatures already made and hardware-rooted; cost lands per-relationship at the consent moment). One MUST binds the interim so the slot stays reachable: **no presentation surface may bake a mandatory stable identifier across verifiers into the wire contract** — the upgrade MUST remain a format addition, never a re-issuance event.

Identifier form is a class rule (normative). Every CC wire-vocabulary identifier — `wrap_algorithm` and the other CC 3.3.2 closed-set strings — is lowercase-ASCII `snake_case`

and is matched by **exact byte comparison**. **Domain-separation constants are a different class and are NOT subject to that rule**: `KEY_GRANT_V1_INFO` (`cewp-key-grant/v1`), the CC 5.3.3.1 STREAM-nonce label `ciris-stream-nonce/v1`, the CC 5.4.2 symbol label `ciris-edge/scope-privacy/symbol/v1`, and the CC 5.4.4 `HPKE_SUITE_ID` are literal KDF/AEAD inputs whose hyphens and slashes are load-bearing bytes; they MUST be used exactly as written and MUST NOT be snake_cased, and they MUST NOT be compared against vocabulary identifiers.

Epoch index is monotonic, per-`stream_id`, greenfield — a **separate addressing axis** from `key_grant.rotation_chain`:

Axis	Addressing	Where it lives	Supersession
Content-addressed grant rotation	`(at_rest_sha256, attested_key_id)` (the recipient)	the substrate's grant store, indexed on the pair	`rotation_chain`, an informative trail of the prior `key_grant_ids`; no row supersedes or removes a wrap (CC 3.3.2)
Stream/epoch-addressed grant rotation	`(stream_id, epoch[, recipient])`	`federation_stream_chunks(stream_id, seq)`	The same `rotation_chain` trail **on the new axis**; a rotation is a new row under a new axis identity

Epoch triggers carry the forward-secrecy guarantee — once a member leaves, subsequent content is sealed under a key they cannot derive:

Trigger	Behavior	Forward-secrecy implication
Member removal	**MANDATORY rotation** (coalesced per below; exempt for ungated public broadcasts per below) — the forward-only-unsubscribe enforcement	Subsequent epochs sealed under a DEK the removed member doesn't have
Member addition	Epoch advances — an MLS Add is a commit, so the exporter secret and every per-stream DEK derived from it change; what a joiner receives is the tier's rule (CC 4.4.3.4.1), and catch-up exists only for a **kept** stream	A new member derives the current epoch's DEKs from the group and no earlier epoch's; for a kept stream the earlier epochs reach the joiner as `key_grant:stream:v1` wraps, bounded by the catch-up bound below; an unkept stream has no catch-up by design
Time / bytes	Optional hygiene rotation	Operator policy; default off

Removal coalescing. At broadcast scale, naive per-removal rotation is unaffordable: with a large audience and ordinary churn, one rotation per departure makes the flat-unicast cascade cost more than the content fan-out itself. The substrate therefore MUST **coalesce removals**: all removals admitted within one STH cadence window **T (default 2 s, the STH cadence grain of CC 5.3.3.3: a producer-signed root every T; a witness cosign per epoch)** are batched into a **single** epoch rotation covering the whole batch. This caps the rotation rate at 1/T **regardless of churn** ($\leq$ 1,800 epochs/hr at T = 2 s), and bounds removed-viewer exposure to $\leq$ T — the same grain the stream's signed roots already run at. A removed member's exposure window is therefore $\leq$ `T`, never "until the next scheduled rotation."

Public-broadcast exemption (normative; generalised to rosters by CC 3.2 `public` open admission). A `live_stream` whose roster is **ungated** — open admission on a `public` community (CC 3.2) AND grants issued to any requester without an admission ceremony — carries **no confidentiality claim against departed viewers** (anyone, including the departed viewer, can re-subscribe and receive the current DEK on request). For such streams, member

removal MUST NOT force rotation (it buys nothing and costs the full cascade); the time/bytes hygiene rotation and the CC 5.3.3.1 nonce-space bounds still apply. Rotation-on-removal remains **MANDATORY for every gated roster** (bounded membership or an admission ceremony: every roster that is not open) — there the DEK *is* the access-control boundary and the forward-only-unsubscribe guarantee is real.

Rekey conforms to MLS TreeKEM (RFC 9420, normative). The epoch-DEK rekey on member change is the MLS TreeKEM construction: an O(log N) path rekey, the commit signed once, with RFC 9420 blank-node / unmerged-leaf handling and parent-hash tree integrity. The hybrid KEM (X25519+ML-KEM-768) is the MLS ciphersuite's HPKE KEM. **One DEK per `(stream_id, epoch)`, two distributors (normative).** For the live participants of a call the group's MLS **exporter secret for that epoch is the epoch DEK** — MLS is the rekey and live distributor, and there is no second key. The same DEK is wrapped as `key_grant:stream:v1` rows (CC 3.3.2, one per recipient, `wrap_algorithm: v2`) for late joiners, recordings and anyone the audience admits — **only for a stream that is kept**: one that is recorded, or whose `history_on_join` is `full`. An unrecorded live call wraps nothing: its DEK exists only inside the live group, so an ordinary call keeps the forward secrecy MLS gives it and leaves no durable wrap that could later open it.

Per-stream DEK under the group exporter (normative). "One DEK per `(stream_id, epoch)`" is read literally: the epoch DEK of a `live_stream` is **derived** from the group's MLS exporter secret for that epoch, with an **empty exporter context** so every implementation derives the same bytes, and the stream bound by a KDF after it:

```
DEK(stream_id, epoch) = HKDF-SHA256( ikm  = MLS-Exporter(label = "ciris-stream-dek/v1", context = "",
    length = 32),
                                     salt = "",
                                     info = b"ciris-stream-dek/v1" || stream_id_utf8 || epoch_be8,
                                     len  = 32 )
```

One label serves both the exporter call and the `info` tag; `epoch_be8` is the `u64` epoch big-endian, the CC 5.3.3.1 encoding, and no length prefix is needed for the reason that section gives (fixed-length tag, fixed 8-byte suffix). A `key_grant:stream:v1` wraps **this** per-stream DEK and therefore opens exactly one stream; the exporter secret itself is never wrapped, so one participant's kept stream never opens another participant's unkept stream in the same epoch. "No second key" above means no second *distributor*; a derivation under the one MLS secret is not a second key. Conformance is proven by a known-answer test set published with CIRISVerify; the 5.3.3.1 nonce prefix is unchanged and already keys on the epoch DEK. **Kept, changed mid-stream (normative):** `history_on_join` is set per stream by its producer (CC 2.1); a call is N such streams and no room policy makes another producer's stream kept. Turning kept on mid-stream wraps the current and later epochs; earlier epochs' DEKs were never wrapped and are gone. Turning it off is forward-only: wraps already issued are not unmade (`withdraws` on a `key_grant` is refused, CC 3.3.2), and wrapping stops from the next epoch. **"Recorded" is not a field:** a stream is kept when its genesis envelope carries `history_on_join: full` or when `key_grant:stream:v1` wraps exist for it; the recording is the after-the-fact CC 3.3.13 `video` blob with `derived_from: stream_id`, and any retention promise rides the producer's `consent:deletion_sla:*`.

Mechanism by content class, never by deployment (normative). Membership-group key agreement is RFC 9420 MLS (TreeKEM, ciphersuite 0x004D X-Wing), whose exporter keys scope two things and only two: **addressing** — the record secret and the destination secret of CC 5.4.1 and CC 5.4.6 — and **live call media** — the stream DEK seed exported under `ciris-stream-dek/v1`, then the HKDF over the label ‖ `stream_id` ‖ `u64_be(epoch)` above.

Content at rest — chat bodies, files, kept or recorded streams — is sealed under content DEKs carried by a flat per-recipient `key_grant` cascade (CC 3.3.2) on three axes: `key_grant:epoch:v1`, the community epoch DEK, rotated by an epoch bump on removal; `key_grant:stream:v1`, one DEK per stream and epoch, for chunk DAGs and kept audio and video; `key_grant:content:v1`, self and family whole blobs. The MLS group never keys a chat body. Which mechanism applies is fixed by the content's class and is not a deployment setting: the tree's advantage is one-to-many commit aggregation among live members, the flat cascade is one grant per recipient over whatever delivery the transport offers, and neither displaces the other. The split is **wire-invisible**: tree position rides the opaque `key_grant` payload; no schema migration.

Forward secrecy is forward-only by default (CC 4.5.12.1, rotation on departure). **Post-Compromise Security (PCS)** is **not provided by default**. A deployment MAY enable it by requiring periodic TreeKEM key-updates (member self-updates), after which a compromised current member heals on the next commit; a deployment that does not stays forward-only.

Catch-up bound: `min(operator depth cap, chunk-eviction horizon)` — the depth cap is an operator setting, not a substrate constant. Three distinct windows that are NOT conflated: chunk-eviction horizon $\neq$ the holding claim's 72 h liveness (CC 3.1.3.3) $\neq$ grant durability. A catch-up request against an evicted epoch returns `ContentMiss` — **fail-honest, no silent gap** (the fail-honest invariant). Operators MUST ship the depth cap **with** the cascade, else the per-rekey Contribution count is unbounded (10^6 for a million-recipient roster).

5.2 Structural invisibility: no holding claim leaves a bounded cohort

The strongest privacy guarantee in the system is the one the operator cannot switch off: self and family content never emits the attestation that would tell the rest of the network it exists, because the wire format cannot carry it. When a Contribution carries `cohort_scope: self` OR `cohort_scope: family`, the substrate MUST NOT emit any holding claim observable outside the cohort — the `custody:ack:v1` report is placed at the content's scope and replicated to members only (CC 3.1.3.3); the content's bytes are delivered to admitted members of the relevant self-collective (CC 3.3.6 `identity_occurrence`) or family (CC 3.3.4) via the at-rest encryption flow, NOT via the public holder-discovery directory.

The privacy property is structural, not policy:

- A non-member peer cannot issue `ContentFetch` for the bytes because no holding claim it can observe names a holder.
- A non-member peer cannot even *discover* the bytes exist via the substrate — the only attestations referencing them are scoped to the self-collective / family and never federate beyond it.
- This is **structural invisibility** at the wire-format level: privacy emerges from format constraints, not from operator policy or legal undertaking.

> ***Scope (normative):*** *structural invisibility buys* ***content-holding confidentiality only*** *— it is NOT relationship-existence, metadata, traffic-analysis, or unobservability privacy. The bounding non-goals are stated canonically at CC 1.13.3.1; do not represent CEG as providing the stronger properties.*

Substrate enforcement:

```
On admission of a Contribution C with cohort_scope in {self, family}:
    # (1) STRUCTURAL INVISIBILITY -- UNCONDITIONAL:
    substrate MUST NOT emit a holding claim for C's evidence_refs bytes outside the cohort
    substrate MUST NOT propagate C beyond the self-collective / family scope
        via any other directory or discovery surface
    # (2) AT-REST ENCRYPTION -- the CC 4.4.3.4.1 cascade, always on (CC 4.4.3.4):
    recipients := if cohort_scope == self:   all current identity_occurrences of C.attesting_key_id
                  if cohort_scope == family: all current members of family per C.family_id
    FOR each recipient r:
        kem := resolve_encryption_keys(r.key_id)    # current occurrence's encryption_pubkeys (CC 3.3.6.1)
        IF kem is None OR kem.ml_kem_768 invalid:
            # FAIL-SECURE: no valid v2 wrap target => EXCLUDE r from the grant.
            # MUST NOT fall back to plaintext or to wrap_algorithm v1.
            # NOT SILENT: emit hard_case:recipient_excluded:{scope_key_id}
            #   (CC 3.4.4, which defines the closed reason-set) INTO the self/family
            #   scope -- recipient r, reason, + the skipped Contribution's envelope
            #   ref. Scoped to the cohort; never federates beyond it (CC 5.2
            #   invisibility preserved).
            skip r   # content stays encrypted + unreachable to r until r registers encryption_pubkeys
        ELSE:
            substrate MUST wrap C's DEK via key_grant (CC 4.4.3.4.1, wrap_algorithm v2)
                to kem.{x25519, ml_kem_768}
```

Two layers, not one. (1) **Structural invisibility** — holding claims that never leave the cohort + non-propagation beyond scope — is the **unconditional** guarantee: no discovery attestation federates. (2) **At-rest encryption** — the CC 4.4.3.4.1 DEK cascade — is **defense-in-depth** (against local-disk forensics, the host operator, or the cloud-substrate operator); it is always on — a body written at a self, family, community or affiliations tier is sealed by the substrate, and no operator switch turns that off — and it uses `wrap_algorithm: v2` (hybrid PQC, CC 4.4.3.4.1), never v1. At those tiers the only plaintext bodies are those of authorized infrastructure communities and public rooms, which CC 4.4.3.2.1 places at federation scope, holding claims included.

Composition with the at-rest encryption flow: persist wraps the DEK (`wrap_algorithm: v2`) under each currently-admitted `identity_occurrence`'s `occurrence_key_id` (self) or each `member.key_id` in the named family's roster (family). New occurrence / new family-member admission triggers retroactive `key_grant` emission for all extant `cohort_scope: self|family` content (the "I bought a new phone and want my history" / "I added Carol to the household" flows from CC 3.3.4 worked example).

Recipient encryption-key resolution + fail-secure exclusion. The wrap target is **not** a recipient's signing key. `wrap_algorithm: v2` needs the recipient's `{x25519, ml_kem_768}` **content-KEM** keys, which the recipient self-certifies via its `identity_occurrence.encryption_pubkeys` (CC 3.3.6.1); the substrate resolves them by `resolve_encryption_keys(key_id)` = the recipient's current (non-superseded, within-`valid_until`) occurrence → its `encryption_pubkeys`. Because this layer **mandates v2**, a recipient whose current occurrence carries **no valid ML-KEM-768 key MUST be fail-secure *excluded*** from the grant — the content remains encrypted and unreachable to it; the substrate MUST NOT fall back to plaintext or to `wrap_algorithm: v1`. To be an at-rest-encryption recipient, an identity MUST have a federation-present occurrence carrying `encryption_pubkeys`. This is the non-maleficence / fail-secure default: a missing key denies access, never downgrades the protection.

Exclusion MUST NOT be silent. A bare `skip` makes a fail-secure exclusion indistinguishable from "the family went quiet" — a soft-censorship vector for a buggy or malicious substrate, and a contradiction of the spec's own `hard_case:*` attestability grain. On every fail-secure skip the substrate MUST emit **`hard_case:recipient_excluded:{scope_key_id}`** (CC 3.4.4, which defines the closed `reason` set) **into the affected self/family scope itself** — carrying the excluded recipient's `key_id`, a `reason`, and the skipped Contribution's envelope ref — so

the excluded member (who still sees cohort-scoped attestations) has something to audit and remediate. The event is cohort-scoped: it MUST NOT federate beyond the self/family (the CC 5.2 invisibility promise is preserved; the *fact* of the family's content is not leaked by its exclusion events).

Locality dividend: `cohort_scope: self|family` content is the bulk of daily activity (family photos, personal notes, in-household device chatter), and that bulk never federates. Operators do not configure this; the wire format enforces it.

Boundary cases:

- One rule at every tier (CC 6.1.5.3): `cohort_scope: community | affiliations` content's holding claims reach the community's members only; `federation` (Commons) content's holding claims are at federation scope and are the holder directory. Self/family is not a special case but the audience rule applied to the smallest roster. Within every cohort, holding claims, custody reports, target replication and anti-entropy run exactly as at every other tier.
- A `cohort_scope: self` Contribution that is later promoted via `supersedes` to `cohort_scope: community` emits its holding claims at the new scope, on the NEW Contribution, at promotion time. The original `cohort_scope: self` Contribution's bytes remain structurally-invisible at federation; only the promoted scope's bytes propagate.
- `cohort_scope: self` content with `subject_key_ids` containing a non-self party (e.g., a private note Alice writes ABOUT Bob) is admitted and stays in Alice's self-collective; Bob does NOT receive a key_grant unless Bob is also in Alice's self (not the case for two distinct identities). Bob's CC 2.3 subject-side revocation authority over the note still composes, but the bytes never reach Bob without Alice's explicit re-emit at a higher cohort_scope including Bob.

5.3 Endpoint shapes

This section specifies the endpoint shapes below for the discovery + cosigning surfaces. Wire-format consumers read these.

5.3.1 STH cosigning + witness directory

The witness directory is where transparency becomes accountable: independent witnesses cosign the log's tree heads so a producer cannot show one history to one party and a different one to another. Consumers verify cosignatures and the witness quorum; CEG's emission half:

`POST /v1/transparency/sth/cosign` (public)

Witness posts cosignature on `(tree_size, root_hash, signed_at)`. Registry verifies hybrid Ed25519 + ML-DSA-65 against witness pubkey in directory; persists on success.

Request body:

```
{
  "tree_size": <int>,
  "root_hash_sha256_hex": "<64-char-lowercase>",     // per CC 2.6.3
  "signed_at": "<rfc3339_canonical>",                // per CC 2.6.2
```

```
  "witness_key_id": "<string>",
  "ed25519_signature_b64": "<base64>",
  "mldsa65_signature_b64": "<base64>",
  "consistency_proof_root_hash_sha256_hex": "<64-char-lowercase>",
  "consistency_proof_tree_size": <int>,
  "consistency_proof_path_b64": ["<base64>", ...]
}
```

Canonical bytes (witness MUST sign these):

```
canonical = sha256(
    "ciris.sth_cosign.v1\n" ||
    "tree_size=" || decimal_no_leading_zeros || "\n" ||
    "root_hash_sha256=" || sha256_hex_lowercase || "\n" ||  // per CC 2.6.3
    "signed_at=" || rfc3339_canonical                       // per CC 2.6.2
)
```

Ed25519 over `canonical`; ML-DSA-65 over `canonical || ed25519_sig` (bound payload).

`POST /v1/transparency/lineage-head/cosign` (public) — trust-root lineage heads

The same directory witnesses **trust-root lineage heads** (CC 3.2 T6): the lineage's signed roster record at a version, which T6 makes the head — named by the SHA-256 of its signed bytes. A witness posts its cosignature; the Registry verifies hybrid Ed25519 + ML-DSA-65 against the witness's key record in the **charter's** witness directory for that lineage (never a directory of its own), refuses a witness that is not independent of the lineage under T6 at the cosignature's `signed_at`, rejects a `signed_at` more than 5 minutes from its own clock at receipt (the CC 2.6.7 bound the STH route already applies, so a witness can neither pre-date nor post-date a re-commitment), and persists on success.

Request body:

```
{
  "lineage_key_id": "<community_key_id or family_key_id>",
  "head_digest_sha256_hex": "<64-char-lowercase over the signed roster record at that version>",
  "prior_head_digest_sha256_hex": "<64-char-lowercase -- the head this witness last cosigned for this
     lineage; empty on its first>",
  "signed_at": "<rfc3339_canonical>",
  "witness_key_id": "<string>",
  "ed25519_signature_b64": "<base64>",
  "mldsa65_signature_b64": "<base64>"
}
```

Canonical bytes (witness MUST sign these; the descent claim is inside the preimage, so a relayed cosignature proves which link it attests):

```
canonical = sha256(
    "ciris.lineage_head_cosign.v1\n" ||
    "lineage_key_id=" || lineage_key_id || "\n" ||
    "head_digest_sha256=" || sha256_hex_lowercase || "\n" ||
    "prior_head_digest_sha256=" || sha256_hex_lowercase_or_empty || "\n" ||
    "signed_at=" || rfc3339_canonical
)
```

Ed25519 over `canonical`; ML-DSA-65 over `canonical || ed25519_sig`. The domain label is distinct from `ciris.sth_cosign.v1`, and the byte layouts differ, so a head cosignature can never be replayed as a tree-head cosignature, nor a cosignature as a head, nor the reverse.

Descent, re-commitment, and the two evidence tiers (normative). A witness MUST verify, before signing, that the head **descends from, or equals,** the prior head it cosigned for that lineage (equality is the T6 cadence re-commitment — the witnesses' duty, not the founders'

— and MUST be accepted; descent is the `prev_head_digest` chain, each intermediate head's quorum checked against its parent's fold and its cosignatures against its parent's directory; on a witness's first cosignature the prior is the lineage's genesis head from the bundle). The Registry MUST reject a cosignature that names a prior it did not record for that witness, or a head that neither equals nor descends from it → `LINEAGE_HEAD_INCONSISTENT` (CC 5.3.6.1); that is the malformed case. Two other cases are evidence, not errors. A head carrying a signature valid under the lineage's quorum at that head's instant that does not descend from the lineage's *witnessed* head is a **fork attempt**: the Registry MUST record it once per `(lineage_key_id, fork point)` — the nearest ancestor on the witnessed chain — MUST NOT cosign it, counts and does not store further attempts off the same point, and raises `hard_case:lineage_fork_attempt` (CC 3.4.2) — it never freezes anything. Two heads that each reach the charter's `witness_quorum` are **equivocation**, `hard_case:lineage_equivocation`, and freeze head adoption under T6; because the quorum is a strict majority of the directory, at least one witness signed both, and the two cosignatures name it. A lineage whose newest counted cosignature is older than its charter's `witness_cadence_secs` is *silent*: reported on the trust surface as a liveness signal under CC 3.2 T4, never a validity leg — its effect is on *attaching* (T4a), where a silent lineage becomes unattachable once `attach_window_secs` lapses — the charter's attach window (CC 3.2 T4a), distinct from `witness_cadence_secs`, which is the witnesses' duty and the liveness signal — and attachable again on the next re-commitment.

5.3.1.1 Consistency-proof requirement (normative)

A cosignature only means something if the witness has checked that the new tree is a consistent extension of the one it last saw — otherwise quorum is agreement on a string, not on a history. A witness signing an STH MUST first verify a consistency proof from the prior STH it cosigned (or from genesis if it is the witness's first cosignature against this log). The Registry MUST reject `POST /v1/transparency/sth/cosign` requests that omit the `consistency_proof_*` fields OR whose consistency proof does not verify against the named prior STH. `witness_quorum_met` is therefore "quorum on log consistency," not "quorum on a string."

The cosign request carries `consistency_proof_path_b64[]` (base64 RFC 6962, section 2.1.2, node hashes). The Registry anchors the check against the prior `(tree_size, root_hash)` **it recorded** for that witness — not a root the requester claims — and rejects: missing proof when a prior STH exists or a tree_size behind the witness's prior cosigned STH → `MALFORMED_REQUEST`; a proof that does not reconstruct both roots → `CONSISTENCY_PROOF_INVALID` (CC 5.3.6.1). A witness's first cosignature is exempt ("from genesis").

5.3.2 Transport substrate for byte-level content

Wire-format Attestations carry claims; they don't carry bytes. When a claim's `evidence_refs[]` cites a SHA-256-addressed blob (e.g., an installer binary, a config file, an adapter package per `agent_files:*` per CC 3.1.7 / CC 3.1.10), the bytes travel via Edge transport substrate: `MessageType::ContentFetch` + `ContentBody` + `ContentMiss`. Holder-discovery via the `custody:ack:v1 here` rows at the content's scope (CC 3.1.3.3); node-mode peers serve the bytes. Attestation envelope shape unchanged; SHA-256 in `evidence_refs[]` becomes universally resolvable to bytes through the substrate.

The durable trace (normative). The substrate's duties for an agent's reasoning trace follow the anchor rule of CC 1.13.6: a trace is durable only when it is anchored on the record of an act. Every execution path that produces an external effect — every utterance, every tool

call, every adapter-mediated emission — MUST close with a terminal result row (the closing action record of the agent's pinned trace schema, CC 3.1.5), and the trace it anchors is carried as `trace:complete:v1` (CC 3.1.5) or its post-scrub `trace_summary:{kind}`. An in-flight trace that reaches no terminal row is working state, not a record: the substrate MUST purge it after a declared, bounded age (the orphan sweep) and MUST export a count of the traces it purges, so an auditor can see how much is discarded while the contents stay, by design, unavailable. A substrate MUST NOT be read as having lost evidence when an unacted deliberation ages out; the orphan sweep can never erase evidence of a consequential act, and if it ever did the defect lies in the action-emission path, never in the purge. Until the totality of the terminal row over effect-producing paths is tested, the invariant MUST be represented as a named wager wherever the control is claimed, and the register carries its status. An operator needing interrupted-path forensics MUST enable a bounded, privacy-scoped incomplete-receipt plane as an explicit opt-in; it is not the default, and this document does not claim one exists on every surface.

5.3.2.1 Holder directory TTL + ContentMiss feedback

A `custody:ack:v1 here` report is live for **72 hours** from its instant (CC 3.1.3.3); after that the holder reads *unknown*. Consumer policy MUST attempt at most 2 holders in parallel and accept the first successful full-SHA verification. On `ContentMiss` (holder no longer has the blob) the consumer MUST NOT withdraw the holder's report — only its signer retracts a holding claim (CC 3.1.3.3 rule 4) — it records the miss and downweights the holder in its own holder resolution; the holder's own next acknowledgement reports `none`.

The cap binds the whole-object fetch (normative). "At most 2 holders in parallel" bounds the fetch in which each holder returns the whole object and parallel attempts duplicate the download. A chunked object (CC 5.3.3.1) is fetched chunk by chunk, each chunk verified by its own seal and committed root, and the same rule applies at chunk granularity: no chunk is requested from more than 2 holders in parallel. How many distinct chunks a consumer keeps in flight with one holder is the implementer's, and the `ContentMiss` downweight above applies per holder on either path.

5.3.2.2 Consent revocations are NOT local-tier-eligible

A user's withdrawal of consent is the one signal federation peers rely on to stop propagating that user's data — so it can never be left unsigned and invisible to them, even briefly. The rule is **local-tier signature deferral** — self-attestations skip the hybrid Ed25519 + ML-DSA-65 signature path locally and only sign at federation-tier promotion. This is sound for **producer-only-authority self-attestations**. **The discriminator is *whose act the row is*, NOT whether `subject_key_ids` is empty**: a row is local-tier-eligible iff it is the producer's own act under its own authority. Such a row MAY name a subject in `subject_key_ids` per CC 2.3.1 — e.g. `observed:user:{hash}:*`, `epistemic:about:{key}:*`, a self-`consent:partnered:{user}`, or the CC 2.3.4 self-`identity:current` with `subject_key_ids=[self]` — and remains local-tier-eligible, because the row is the producer's own act; every listed subject keeps its withdrawal authority over it under CC 2.4.1.1 rule 2, and a listed subject's later `withdraws` or `consent:state:revoked` is that subject's **own signed Contribution**, never a durable local row. The single carve-out is below: a **subject's own revocation** over a Contribution that names it.

Consent revocations from subjects MUST NOT use the local-tier deferral path. When a Contribution carries non-empty `subject_key_ids`, any subsequent `consent:state:revoked` emission OR `withdraws` admitted under CC 2.4.1.1 rule 2 or 3 from a subject in that set MUST promote to federation-tier within **the window (24 hours by default, operator-tunable)**.

Rationale: subject-side revocation is the wire-format observability primitive that federation peers depend on to honor consent. If a user revokes consent in the local-tier scope of one agent's substrate, and that revocation is unsigned + unpromoted for an extended window, other federation peers continue propagating the user's data — exactly the failure mode this closes.

Substrate emission: substrate MUST emit `hard_case:consent_revocation_promotion_overdue` when a subject-side revocation has been local-tier for longer than the window without federation-tier promotion. LensCore composes `detection:consent:promotion_delay_pattern` on top (operator monitoring; not a slashing trigger on its own).

Composition with the self-attestation pattern: the CEG-native agent's `consent:partnered:{user_key}` self-attestation (producer-side stance) MAY ride local-tier; the user's `consent:state:revoked` against the agent's stance Contribution (subject-side) MUST NOT. This preserves the cardinality win of deferral while closing the leak window.

The consent tail: transit, not rest. The "no local subject-revocations" rule and the consent-SLA promotion gate above are the **same invariant viewed from the two gates**, not a contradiction. A subject-side consent revocation — a subject-emitted `consent:state:revoked`, or a `withdraws` admitted under CC 2.4.1.1 rule 2/3 from a subject named in the target's `subject_key_ids` — is **federation-tier by classification**. It MAY *transit* the local-tier write path while in flight (the producing occurrence's substrate accepts the unsigned envelope before promotion), but it MUST NOT *rest* there: a subject revocation is **never a durable local row**. Its only conformant terminal state is *promoted* (federation-tier, hybrid-signed per CC 5.3.2.4.3); *overdue-flagged* is a recorded breach of the window, not a resting state, and *settled local* is never reached. The substrate MUST drive it to federation-tier promotion within the window; the local tier is a momentary staging buffer for it, not a resting place.

The overdue emission records the breach. When the window lapses without federation promotion, the substrate MUST emit `hard_case:consent_revocation_promotion_overdue` (as required above). This emission is not merely observability — it is the signal the consent-promotion path keys off: the consent-promotion gate and the CC 5.3.2.4.3 `local` $\Longrightarrow$ `caller is the producing occurrence` read-gate read the **same** overdue state. Because both gates key off one shared signal, a subject revocation can never be simultaneously (a) invisible to federation peers under the local read-gate and (b) un-flagged by the consent-promotion gate. Keeping the two gates reading one state is exactly what closes the de-synced-gates case (CC 5.3.2.4.3).

5.3.2.3 Cross-region merge intents: CEG-declared per subject_kind (normative)

With operational data joining the CEG-native replication stream (CC 3.3.9), the substrate runs **more than one merge policy**. The policy is a **normative property of the subject_kind, declared here** — the substrate reads and dispatches on the declaration; it MUST NOT infer policy per record type (the substrate enforces declared merges; it does not invent policy).

subject_kind(s)	Merge intent	Semantics
organization, org_membership	lww_skew_bounded + withdrawal_forward_only	Stable-id grouping (CC 3.3.9); an admitted withdraws (deactivation) is forward-only — a later non-withdrawn write does NOT resurrect; else latest asserted_at wins; tie-break smallest attestation_id (CC 3.5.1). The forward-only here is the **lightweight authz flag**, NOT the CC 4.4.3.4 DEK-cascade crypto path — Commons tier, no DEK exists.
the licence (licensure: {authority_id}, CC 2.4.1.2.1), written under the affiliation record's M-of-N quorum (CC 4.4.3.2.8, CC 3.3.9)	monotonic_quorum (the revocation merge below, on license_id)	Admission anti-rollback first: a write whose revision decreases never enters the merge. Then the MergeBallot comparator: quorum_weight → signed timestamp → content hash. Quorum-above-time is deliberate — it neutralizes timestamp front-running on the records where it matters most. More-restrictive state wins on conflict: revoked > suspended > active.
revocation + the three membership-revocations	monotonic_quorum (the original instance)	The same anti-rollback and comparator as the licence row. key:revocation:v1 merges on its digest subject revoked_key_sha256_ed25519_raw (CC 2.4.1.1): the per-subject anti-rollback ordering keys on the digest. The three membership revocations merge on the member key_id, since roster rows are addressed by key_id.
keys / attestations / occurrences / families / communities / location_proofs	Content-addressed idempotent admission	Same content → same envelope_hash → dedup; rotation collisions rejected non-destructively.

Skew-bounded admission (normative). For every subject_kind merging by lww_skew_bounded, the substrate MUST reject at admission any envelope with asserted_at > now + tolerance, where tolerance is the CC 2.6.7 clock-skew bound (±5 minutes; the existing CLOCK_SKEW_VIOLATION error class). Without this, a signer with a forward-skewed clock future-dates asserted_at and wins LWW indefinitely; with it, a forward-dated write can win for at most the skew window. This matters precisely because org_membership carries role: OrgAdmin — unbounded LWW on authz data is a role-escalation surface.

The two quorums (restated from CC 3.3.9 — different layers, different stewards): the **steward-signature admission quorum** (M-of-N signature set over identical JCS bytes; the CC 3.2 founder-quorum machinery at admit — Verify's layer) is NOT the **region merge quorum** (quorum_weight, the MergeBallot tier-1 ordering above — the substrate's layer). The substrate's merge logic never counts steward signatures; Verify's admission check never orders merges.

5.3.2.4 The attestation tier model: local-tier write, query, promotion (normative)

Pins the **shared attestation surface** every implementation of the row store and the relay writes, queries and promotes through, so it is analyzable + modelable as one contract. The substrate exposes the *methods*; CEG pins the *wire/conformance semantics* every implementation MUST agree on. **No change to the 1+4 surface** (CC 1.7) — a "tier" is recorded state on

an attestation, not a new primitive. The tier split serves the same fail-secure cost discipline as streaming: cheap, unsigned writes stay private and local until the costly hybrid signature is paid exactly once, at the moment authority actually crosses to the federation.

5.3.2.4.1 Local-tier eligibility: the discriminator is *whose act the row is*, not subject-set emptiness

A write is local-tier-eligible iff **it is the producer's own act under its own authority**. The discriminator (*whose act the row is*, NOT empty `subject_key_ids`), the producer's-own-act-with-named-subject examples, and the single carve-out (a listed subject's own `withdraws` under CC 2.4.1.1 rule 2/3 or subject-emitted `consent:state:revoked`, which is that subject's own signed Contribution and MUST go signed / promote within the CC 5.3.2.2 window, never a durable local row) are defined canonically at CC 5.3.2.2. Tier-specific addition: `witness_relation` MUST be `self` for any local-tier write.

5.3.2.4.2 Promotion: `local` → `federation` (the deferred-signature moment)

Promotion computes the hybrid signature and flips the row federation-visible. It is **idempotent** (promoting a `federation` row returns it unchanged), and at the promotion instant the tiered-scope promotion (CC 4.4.3.3.1) and the holding claim at the new scope (CC 3.1.3.3) fire exactly as for any federation write — **promotion *is* the federation-emit moment**.

Canonical bytes — `JCS(envelope)` per CC 2.6.1 / RFC 8785:

- The signature MUST cover `JCS(contribution_envelope)` — the **identical** canonical bytes any natively-federation attestation signs. A promoted row is therefore **byte-indistinguishable on the wire from one born federation-tier**; there is no "was-promoted" marker in the signed bytes.
- **Substrate columns are NOT in the canonical bytes.** `tier`, `promoted_at`, and any other storage bookkeeping are substrate state, never part of `JCS(envelope)`. Only the CC 2.1 envelope member set is canonicalized.
- **CC 2.6.1 omit-vs-materialize is load-bearing here.** Promotion MUST canonicalize the **exact member set the producer committed at local-write time** — a field *omitted* at local write MUST NOT be materialized at promote, and vice-versa, or the recomputed bytes diverge from what a peer verifies and the hybrid sig fails. The substrate serializes the stored row → the committed envelope → JCS → sign; it MUST NOT re-default.
- Registry owns the exact envelope member set (CC 2.1 + the CC 2.6.1.2 catalog); the verifier recomputes the identical JCS bytes. Do NOT use the verifier's internal length-prefixed `signing_bytes` framing — that is for verify-to-verify primitives that never cross the four-impl boundary as JSON; a promoted attestation is the opposite.

5.3.2.4.3 The two tiers

Tier	Signature	Federation-visible	Read-visibility	Written by
`local`	MAY be absent (deferred per CC 5.3.2.2)	**No**	**only the producing occurrence** (self-read); every other caller — even an authorized family/ community peer — sees nothing	local-tier write
`federation`	hybrid Ed25519 + ML-DSA-65 **present**	Yes	per CC 2.1 `cohort_scope` + the CC 5.2 invisibility rule	direct signed write OR `local` → `federation` promotion

Invariant (substrate MUST enforce): `tier = federation` $\Longrightarrow$ `hybrid signature present`. Nothing crosses to federation-visible unsigned. A `local` row is **labelled** local and MUST NOT be served as federation-authoritative (fail-honest). The read-gate is **orthogonal** to `cohort_scope`: it is an additional filter (`local` $\Longrightarrow$ `caller is the producing occurrence`), composing with the CC 5.2 target-membership predicate. The three failures this closes: a local row leaked to a non-self caller, an unsigned local row served as authoritative, the two gates de-synced.

5.3.2.4.3.1 The PQC half is MANDATORY at admission: no classical-only, no hybrid-pending accommodation (normative)

The CC 5.3.2.4.3 invariant `tier = federation` $\Longrightarrow$ `hybrid signature present` is **enforced at the admission gate**, and "present" means **verified**: every federation-tier admission gate MUST check **both** halves — Ed25519 over `JCS(envelope)` **and** ML-DSA-65 over `JCS(envelope)` $\|$ `ed25519_sig` (the CC 3.1.2.1 bound-payload form) — and MUST **reject** a federation-tier Contribution that carries only the classical half. This binds **all operational-authority admission gates**, explicitly including `operational_admit` (key_grant / partner / org-membership / license writes), the CC 3.3.6.2 `transport_destination` binding, and the CC 3.2 founder-quorum gates, under which the affiliation record's licences are written (CC 4.4.3.2.8).

This is an immediate requirement. There is no opportunistic-acceptance window and no fleet-floor or calendar trigger. The rationale is exactly the threat hybrid exists to defend: an adversary holding a future Ed25519 break could forge a grant / binding / licence and have it admitted if the classical half alone sufficed. There is **no `require_hybrid: false` posture** — a verifier's hybrid-required check is always-on, never an operator toggle; the only conformant state for a federation-tier key is "carries a valid ML-DSA-65 half." A key that has not completed PQC wiring is **not conformant for federation-tier emission** and is confined to local-tier (self-read, CC 5.3.2.4.1) until it does. The mandate is at the **federation admission boundary only** — the single place authority crosses; local-tier rows MAY still defer the signature per CC 5.3.2.2.

The mandate binds the durable store + replication path, not only the authority gates. "Federation-tier" means **every** federation-tier attestation, including the bulk **per-trace / store-and-replicate** path — not merely the operational-authority gates enumerated above. A federation-tier trace written to the durable, content-addressed, replicated corpus MUST carry (and a verifier MUST check on ingest) the ML-DSA-65 half exactly as a `key_grant` does; there is **no "operational authority vs testimony leaf" exemption** (federation root and the trace leaf are bound by the same rule). **Content-addressing is NOT a defense against forge-later:** a CRQC-era adversary who breaks Ed25519 mints a backdated trace under a historical key and

hashes **their own** forgery — the content hash matches by construction, so the address proves nothing about authenticity. Because the trace store is *kept for posterity* (it outlives the classical primitive), the per-trace signature is the single most forge-exposed surface in the federation — the crux of a store kept at scale — and the PQC half is mandatory there for the same forge-now / harvest-now-exploit-later reason as the at-rest DEK cascade (CC 5.1 / CC 4.4.3.4.1). A substrate persisting/replicating a federation-tier row whose envelope signature lacks a valid ML-DSA-65 half MUST reject it at the ingest gate — store-then-quarantine is non-conformant.

5.3.2.4.4 Query: open-prefix dimensions, bounded operators (normative)

The uniform read filters on `(dimensions[], valid_at, confidence_floor, subject_key_id?, scope)`.

- **`dimensions[]` is an OPEN-vocabulary set of prefix strings, matched by hierarchical prefix — NOT a closed enum.** Per CC 4.5.1.1 axis-vocabulary discipline, dimension prefixes are open vocab (new families — `consent:*`, `detection:community:*`, `settlement:*` — ship continuously). A closed enum would force a substrate redeploy per CEG namespace addition and break forward-compat. A query for `detection:*` matches `detection:correlated_action:bribery` etc.; an exact string matches exactly.
- **The bounded surface is the *operator set*, not the *vocabulary*.** The apophatic discipline (a fixed, named surface — not an OLAP/graph engine) is preserved by the **closed set of five predicates** above + no caller-composed SQL/projections — NOT by closing the dimension vocabulary. *Open data, closed operators.*
- `valid_at` filters `asserted_at` $\leq$ `valid_at` $<$ `COALESCE(expires_at,` ∞`)`; `confidence_floor` filters `weight` $\geq$ `floor`; `scope` applies BOTH the CC 5.2 target-membership gate AND the CC 5.3.2.4.3 tier gate. Write-side reserved-prefix emitter gates (CC 3.4.7.1) are unaffected — read is gated by scope, not by dimension authority.

5.3.2.4.5 For holistic analysis + modeling (informative)

The tier model is the *same KEM-then-symmetric placement* the streaming model uses (CC 5.3.3), applied to attestations: the **expensive op (hybrid sign) is on the cold path** (promotion, at federation-emit), while the **hot path (local self-attestation write) is O(1) and unsigned**. Cost shape for a node:

- **local write**: O(1), no asymmetric crypto — the agent's steady-state memory/config/consent/identity churn.
- **promotion**: one hybrid Ed25519+ML-DSA-65 sign (the CC 5.3.3-model per-op cost) + JCS canonicalization — only at the federation-emit moment, never per local write.
- **query**: the CC 2.1 scope-filtered read; cost is the predicate + index, independent of tier.
- **observability for modeling**: `local` row count, promotion rate, and `hard_case:consent_revocation_promotion_overdue` count are the three measurable signals; the CC 5.3.2.2 window is the one tunable. A model of a federation's attestation load is therefore *(local-write rate, promotion rate, query rate)* with promotion carrying the only asymmetric-crypto cost — the dual of the streaming model's epoch-rekey tail.

5.3.2.5 Full-SHA verification before consumption (normative)

A CEG-Conforming Consumer (CCC) MUST verify the full SHA-256 of received bytes against the value in `evidence_refs[]` BEFORE handing the bytes to any consumer (Agent loader, Portal renderer, etc.). A holding claim names the blob by its address digest (CC 3.1.3.3); the consumer MUST verify the full SHA of the received bytes regardless of how it found the holder. Bytes that fail the full-SHA check MUST be discarded and the holder MUST be reported per consumer policy (a negative score on the holder; never a `withdraws` of another device's holding claim).

Every blob cited through a descriptor carries its size, and size is checked first (normative). A descriptor that cites a blob from `evidence_refs[]` — installer, config, adapter package, media, any bytes — carries the blob's byte length alongside its digest. A producer always knows it at creation time. **Where the size lives:** an `evidence_refs[]` entry is a bare URI or content-hash string on the frozen CC 2.1 envelope and carries no size; the size rides the **descriptor** whose canonical shape this document already defines — the multimedia Source struct's `{digest, size}` (CC 3.3.13), the live-stream chunk-descriptor list (same section), the per-artifact entries of `SkillImportManifest` and the build manifest, and the `build` descriptor of a build-manifest Contribution, which carries `manifest_size` beside `manifest_hash` (CC 3.1.2.1). This rule binds every blob cited **through** such a descriptor. A blob cited only by a bare `evidence_refs[]` string has no declared size, and a consumer fetching it enforces the CC 2.6.1.3 cap instead — it MUST NOT invent a size field on the envelope, which would be a wire change; a producer that needs the size check on such bytes cites them through a descriptor. The consumer MUST compare the received length to the declared size **before** computing the digest: a mismatch is a refusal without reading the rest, and the descriptor alone lets a receiver enforce caps and refuse a `ContentFetch` that would overrun. Size is not a substitute for the digest, only the cheaper check that runs first. The one descriptor with no total size is a **live stream in progress** (CC 3.3.13 `live_stream`): each chunk and the init segment carry their own, and the recording, once the stream ends, is an ordinary blob with one.

5.3.2.6 The render tier is receiver policy, computed from verified, sniffed bytes (normative)

A CCC MUST decide whether and how to render a blob from the **verified bytes and its own policy, never from a claim in the descriptor**. After CC 5.3.2.5 verification the consumer MUST sniff the leading bytes and require the sniffed essence to **equal** the declared `format` (CC 3.3.13); a mismatch is a **refusal, not a correction**. Where the description is sealed (`sealed_descriptor`, CC 3.3.13), the sniff runs where the bytes decrypt, by the party that opened them, against the `format` inside the seal; a holder that cannot open the bytes does not sniff and does not claim to. The declared `format` / `codec` are inputs to that comparison and to capability checks, never authorisations. This is what makes the set below a property of the *receiver* that a malicious sender cannot widen.

Recommended renderable set (recommended CCC policy — not normative). The intersection of what the widely deployed messengers *emit*, what the common mobile and browser platforms all decode, and what has a memory-safe decoder. A CCC MAY adopt a narrower set; a wider one is its own risk to own, and this Part does not endorse it.

`format` (sniffed)	Tier	Conditions
`image/jpeg`, `image/png` (APNG as PNG), `image/gif`	**A — render**	$\leq$ 16 MB, $\leq$ 33 Mpx; GIF $\leq$ 921,600 px + frame cap; reject bytes after `IEND` / `FFD9`

`format` (sniffed)	Tier	Conditions
`image/webp`	**A — conditional**	≤ 10 MB; node runs a WebP sanitizer; re-encoded on ingest
`text/plain`	**A**	valid UTF-8, ≤ 1 MB, bidi controls rendered visibly
`video/mp4`	**A — conditional**	`codec` present and `avc1.*` (≤ L4.1) + `mp4a.40.2` only; ≤ 100 MB, ≤ 8,294,400 px; node runs a hardened MP4 sanitiser (for example `mp4san`); receiver rejects unlisted tracks
`audio/mp4` (`mp4a.40.2`), `audio/mpeg`	**A**	ID3v2 ignored — never decode `APIC`
AVIF, HEIC / HEIF, JXL, WebM, MOV, MKV, Ogg / Opus, FLAC, WAV, ADTS, Markdown	**B — sender converts to a Tier-A form before hashing; raw arrival = C**	not decoded natively by the common mobile and browser platforms; no memory-safe decoder is assumed
`image/svg+xml`	**C for the bytes — never a client renderer or WebView.** A node MAY emit a PNG rendition (`derived_from`) from a hardened rasteriser (for example `resvg`: no file-href resolver, no system fonts, caps) in an isolated process; that PNG is Tier A	scriptable, with external references and font resolution; no client renderer is admitted. The CC 3.3.13 rendition rule dissolves the B-or-C question: the SVG is C, the node's PNG is A
`application/pdf`	**C — download**; node MAY rasterise pages to attested PNG in an isolated process	JS / Launch actions / embedded fonts; viewer escapes recur
`model_3d` (glTF, USDZ, FBX, splats)	**C — download**; node MAY rasterise a PNG poster in an isolated process	scene-format parsers fail on memory safety recurrently, and a glTF text field can carry script into a 3D viewer
HTML, Office, fonts, archives, TIFF, PSD, BMP, ICO, everything else	**C — refuse**	generic card, external handoff, dangerous-extension block on save

Where enforcement lives. The ingest pipeline that makes Tier A enforceable — size (CC 5.3.2.5: checked first) → full-SHA → sniff (only on verified bytes, CC 5.3.2.6) → memory-safe demux / decode → canonical re-encode → strip metadata → emit rendition + descriptor — belongs on the **node**, so every client renders only bytes the node's safe encoder produced. Decoders for every Tier-A format MUST be memory-safe. Images, SVG and audio decode entirely in memory-safe code; video is "memory-safe code validates container and elementary stream, hardware decodes".

5.3.3 Streaming transport, per-stream logs & delivery receipts

This is the **delivery axis** — the third orthogonal envelope concern alongside visibility (`cohort_scope`) and revocability. CC 2.4's 1+4 primitive set is untouched; this is an endpoint + envelope + composition extension, NOT a grammar change.

Bifurcation:

Half	Cardinality
Observer-share / directed delivery (single Contribution → subscriber-set; no `stream_id`; rides CC 3.3.2 `key_grant` + CC 4.4.3.2 Policy M membership)	N=1 typical; per-subscriber `key_grant`
Media / streaming multicast (`live_stream` chunk-DAG; per-`(stream_id, epoch)` keys; specified by CC 5.3.3.1 / CC 5.1 / CC 5.3.3.6)	N>1; live media keyed from the group exporter, kept epochs carried by flat per-recipient `key_grant:stream:v1` wraps (CC 5.1)

5.3.3.1 Chunk seal + STREAM nonce (normative)

Per-chunk content sealing **uses SFrame's per-frame AEAD construction** (RFC 9605, normative) with the binding and nonce pinned here: the per-frame AEAD model with a `(KID, CTR)` header, here bound as `KID = stream_id` and `CTR = counter`, AEAD = AES-256-GCM, keys derived per MLS epoch (the canonical SFrame+MLS composition, CC 5.1). The STREAM nonce below is CEG's binding profile of SFrame's per-sender nonce derivation. Per-chunk content sealing uses AES-256-GCM (NIST FIPS 197 + SP 800-38D). The 12-byte (96-bit) nonce follows the **STREAM** layout (Hoang, Reyhanitabar, Rogaway, Vizár — *Online Authenticated-Encryption and its Nonce-Reuse Misuse-Resistance*, CRYPTO 2015):

```
nonce[12] = prefix[7] || counter_be[4] || last_flag[1]
```

- `prefix[7]` — derived, NOT transmitted: `prefix = HKDF-SHA256(epoch_dek; info)[0..7]` where the HKDF `info` is the **byte-exact** concatenation: `info = b"ciris-stream-nonce/v1" ‖ stream_id_utf8 ‖ epoch_be8`, with `stream_id_utf8` = the UTF-8 bytes of the stream id as a string — the lowercase hex of the 32-byte stream-id digest the chunk header carries — and **`epoch_be8` = the `u64` epoch as 8-byte big-endian** (`epoch.to_be_bytes()`, consistent with `counter_be`). **This encoding is normative and MUST be byte-identical across producer, substrate, and every consumer** — the CC 5.1 zero-trust-of-host posture has consumers recompute this exact nonce to *open* chunks, so any BE/LE/ASCII disagreement on `epoch` yields a different `prefix` → different nonce → GCM auth-tag failure (silent whole-stream decryption failure), the same hazard class `counter_be` already guards. No length-prefix on `stream_id` is required: the fixed-length tag prefix + fixed 8-byte `epoch` suffix make the parse unambiguous (distinct `(stream_id, epoch)` pairs always yield distinct `info` — unequal `stream_id` length changes total `info` length; equal length splits unambiguously). Matches the `KEY_GRANT_V1_INFO` versioned-context HKDF pattern (`b"cewp-key-grant/v1"`). Per-`(stream_id, epoch)` unique; verifiable by any holder of the epoch DEK
- `counter_be[4]` — 32-bit big-endian; hard ceiling 2^{32}-1 chunks per epoch. Substrate MUST force an epoch roll before wrap. Recommended operational cap: `MAX_CHUNKS_PER_EPOCH` = 2^{24} (~16.7M chunks/epoch) to keep per-epoch state + proof sizes bounded (operator-tunable)
- `last_flag[1]` — `0x01` on the final chunk of an epoch (sealed by `seal_stream` per CC 5.1); `0x00` otherwise. It is set by the epoch terminator and is not a header field: like `prefix`, every holder derives it, from the epoch's seal, and never reads it off the chunk. The distinct nonce on the final chunk gives **truncation + append resistance**: an adversary cannot drop the final chunk and pass off a short stream, nor append past a sealed segment

Cross-epoch counter reset is nonce-safe (normative reasoning): GCM's catastrophic case is reuse of a `(key, nonce)` pair. On epoch roll the DEK changes, so a reset counter lives in a different keyspace — `(DEK_e, nonce=0)` and `(DEK_{e+1}, nonce=0)` are distinct pairs. The enforced invariant is therefore only within a single epoch: counter strictly monotonic, never wraps (guaranteed by the forced roll). Across epochs, reset is free.

Single-sender-per-`(stream_id, epoch)` invariant (normative — nonce-collision safety; as in SFrame/MLS): the `counter_be[4]` space of a `(stream_id, epoch)` is owned by **exactly one** sender — the `stream_id`'s producer. Two distinct senders MUST NOT seal chunks under the same `(stream_id, epoch)` DEK, or their independently-incremented counters could collide into a reused `(DEK, nonce)` pair (GCM-catastrophic). In **group video** (CC 5.3.3.2) each participant emits their **own** `live_stream` with their **own** `stream_id`, so the nonce prefix `HKDF(epoch_dek; "ciris-stream-nonce/v1" ‖ stream_id ‖ epoch)` is per-sender-unique by construction — the same hazard SFrame avoids with per-sender keys, achieved here by per-sender `stream_id`. Substrate MUST reject a chunk-append whose `(stream_id, seq)` collides with an existing sealed chunk (replay/forking guard, CC 5.1).

5.3.3.2 Realtime group communication: composition

The delivery axis was framed for 1:1 observer-share and 1:N broadcast multicast. Realtime **group communication** — group video, voice, desktop/screen sharing, text chat, and topic-scoped channels with sub-channels — is the **same primitive set at N↔N cardinality**. It composes entirely from `community` (CC 3.2) + `live_stream` (the CC 5.3.3 streaming surface) + `chat_message` + member/transport resolution (CC 4.4.3.2.4.1). **Zero new structural primitives** (CC 1.7): the whole realtime-collaboration surface is composition.

The composition map (all generic — no new wire):

Surface	Composes from
1:1 / group video call	each participant emits a `live_stream` (their A/V) scoped to the channel `community`; each subscribes to peers. N↔N = N simultaneous bidirectional streams over a small roster
Voice channel	identical to group video with an audio-only codec; same `live_stream` wire
Desktop / screen sharing	a `live_stream` whose source is a screen capture (1→N within the channel); same chunk-seal + epoch-DEK wire
Text chat	`chat_message` at `cohort_scope: community`, `community_id: <channel>`; threads via `topical_relation: replies_to`
Channel	a `community` (persistent) — its roster gates who can join the call / read the chat
Sub-channels	nested `community` membership (CC 4.4.3.2.5 multi-level pattern): a parent "space" community whose members admit child channel-communities; sub-channel members are a subset of the space roster
Presence ("who's here")	the reachable set (CC 5.3.3.4) — node-local, never an attestation, never logged
Invite / join / leave	community admission ceremony (CC 4.4.3.2.3) — invite = membership proposal (`membership:proposal:v1`); join = admitted member, by a membership widening **on the invitee's own** `membership:acceptance:v1` (CC 3.1.3.2); decline = `membership:decline:v1`; leave = forward-only `withdraws`

Transport profiles: *(normative; extends CC 5.3.3.5)*

Profile	Topology	Transport
Broadcast (1:N, large N)	asymmetric	pull-only `ContentFetch` (CC 5.3.3.5 E2); no relay tree
Realtime small/medium group (calls, huddles, ≤ ~50)	symmetric mesh	**direct Reticulum Links between participants** (low-latency push; no relay tree needed at small N)
Realtime large group (≫50 in one A/V room)	fan-out	pull-only `ContentFetch` (CC 5.3.3.5 E2), as for broadcast; no selective-forwarding relay role

The low-latency realtime profile (direct Reticulum Links) needs no relay tree for small/medium rosters — RNS Links give encrypted low-latency point-to-point natively. The wire is identical to broadcast (chunk-seal CC 5.3.3.1, per-`(stream_id, epoch)` epoch DEK CC 5.1, per-stream STH CC 5.3.3.3); only the transport profile + roster size differ: a live chunk and a stored chunk are the **same sealed object**, so "record a call" is retention policy over chunks that already exist as a DAG, "VOD" is the puller at offset zero, and a licence is a `key_grant` the owner chooses to issue over the same bytes. **PQC is unchanged and mandatory** — epoch DEK wrapped `wrap_algorithm: v2` (X25519+ML-KEM-768) at rest, hybrid KEX in transit.

Ephemeral vs persistent rosters: a persistent channel is a long-lived `community`; an ad-hoc call is an **ephemeral `community`** (short `valid_until`, torn down on last-leave). Same primitive, different lifetime — no special-case "session" primitive.

The breadth confirmation: a full realtime group-communication platform — group video, voice, screen sharing, chat, and channels-with-sub-channels — requires **no addition to the 1+4 structural set** beyond the CC 5.3.3 delivery axis and the `community` membership machinery already locked.

5.3.3.2.1 `codec_id` namespace: realtime A/V chunk codec discriminator (normative)

CC 5.3.3.2.2 makes the realtime chunk payload codec **application-defined and opaque to the substrate**. For realtime **A/V at scale**, a fetching node asks a serving peer for one layer, and the peer sends that layer, **without** either side decrypting the chunks — so the codec's layer-numbering semantics must be a **clear (non-AEAD) discriminator**; no relay reads it. CEG defines a 1-byte `codec_id` namespace so every implementation reads the same meaning. **This is a namespace on the transport-layer chunk header, not a change to the CC 2.1 attestation envelope** — the frozen 1+4 surface and its canonicalization are untouched.

Wire position (normative). `codec_id` (1 byte) + `ChunkLayer { spatial: u8, temporal: u8, quality: u8 }` (3 bytes) = a **4-byte additive block** at `SealedAvChunk` header offset **48..52**, after the 48-byte fixed header. **Clear metadata, NOT inside the AEAD** — a fetching node asks for, and a serving peer sends, one layer by `codec_id`/`ChunkLayer` without touching the inner epoch-DEK seal; no relay reads it (CC 5.3.3.1); tampering causes mis-decode or drop, never a crypto break. No length-prefix is needed — the block is fixed 4 bytes at a fixed offset.

Hex	Codec	Semantics
`0x01`	**AV1 SVC**	Scalable Video Coding — 3 spatial × 4 temporal × N SNR layers; base layer required to decode anything. Production default (WebRTC-native, royalty-free).

Hex	Codec	Semantics
`0x02`	JPEG XS (layered)	Low-latency intra-only; broadcast use case. **Reserved.**
`0x03`	**Symmetric MDC**	Multiple Description Coding — any subset of chunks decodes at proportional fidelity, no base-layer floor. **The substrate design target** (below). **Reserved**; no production encoder is assumed; substrate is MDC-ready.
`0xFF`	Opaque	No scalable-coding semantics. `ChunkLayer` MUST be `{ 0, 0, 0 }`. Default for non-layered streams.
`0x04`–`0x7F`	—	**Reserved** for future standardized codecs (CEG-assigned).
`0x80`–`0xFE`	—	**Experimental / per-deployment** — no cross-federation meaning guaranteed.

`codec_id` lets a receiver know whether `ChunkLayer { spatial: 2, temporal: 2, quality: 0 }` means "SVC base + 2 spatial enhancements" or "MDC quadrant" — without it, layer numbers are ambiguous across codecs. The substrate (Edge) never picks the codec — choice is upstream (agent/server tier); Edge needs only the namespace stable. The variable-depth `SubStreamPath` (MDC tree-path encoding) is reserved for amendment.

MDC-primacy design intent (informative). The user-facing contract the realtime A/V plane targets is *"any node can request a lower-bandwidth stream from peers — as simple as taking every other chunk, down to a blinking dot."* MDC (`0x03`) matches that **symmetrically**: drop any subset → decode the rest at proportionally lower quality, no coordination, no base-layer floor. SVC (`0x01`) is the production default but has a floor (base-layer bytes must arrive) and needs coordination for an "every other chunk" drop. The substrate is **MDC-shaped** (the `ChunkLayer` / `SubStreamPath` model is symmetric-drop-ready) while the production default is SVC; the MDC compression overhead against SVC at equal quality is the accepted cost of symmetric drop semantics.

5.3.3.2.2 Realtime non-A/V data streams (normative scope boundary)

The realtime profile is **not media-only.** Any high-frequency mutable shared state — multiplayer game ticks, collaborative-editing operations (CRDT/OT), live cursors/whiteboard strokes, remote-control input, high-rate telemetry — rides the **same** CC 5.3.3.2 realtime transport (direct Reticulum Links; at scale, pull-only ContentFetch per CC 5.3.3.5 E2) with an application-defined payload codec in place of an A/V codec. The wire is identical: per-`(stream_id, epoch)` epoch DEK (CC 5.1, `wrap_algorithm: v2` PQC), STREAM-nonce chunk seal (CC 5.3.3.1), per-stream STH (CC 5.3.3.3). A data-stream chunk is just a chunk.

Scope boundary (the explicit call):

- **IN scope (transport):** ordered, sealed, authenticated, PQC-encrypted realtime delivery of arbitrary data-stream payloads — covered by CC 5.3.3.2, no addition.
- **OUT of scope (merge semantic):** the conflict-free / convergent-merge logic for shared mutable state (CRDT, Operational Transform, last-writer-wins, etc.) is **application-layer**, not a CEG primitive — consistent with the CC 1.13.4 discipline ("the substrate stores; the wire transports; CEG describes the shape of the claim; consumer policy composes verdicts"). CEG carries the ops; the application converges them. A codified merge primitive, if one is

ever added, routes through the CC 4.5.1 amendment process; none is required, because a realtime collaborative application is buildable on the transport as it stands.

5.3.3.2.3 `SealedAvChunk` wire layout (normative)

The realtime A/V chunk that lands on each RNS Link payload (and the broadcast pull path). **Byte layout (normative):**

```
offset  field                      encoding
0..32   stream_id                  32 bytes (caller-derived: sha256(stream_meta))
32..40  epoch                      u64 big-endian
40..48  chunk_seq                  u64 big-endian
48..49  codec_id                   u8  (CC 5.3.3.2.1 namespace)
49..50  layer.spatial              u8
50..51  layer.temporal             u8
51..52  layer.quality              u8
52..    double_sealed_ciphertext   remaining bytes
```

- `CHUNK_HEADER_LEN = 48` (the `stream_id`+`epoch`+`chunk_seq` fixed header); `CHUNK_CODEC_LAYER_LEN = 4` (the `codec_id`+`ChunkLayer` block).
- **The block is always present; no length-disambiguated compatibility (normative).** The 4-byte `codec_id` + `ChunkLayer` block is part of every chunk: the header is 52 bytes, and a wire shorter than 52 bytes, or whose ciphertext does not carry its AEAD tag, is refused as malformed. Length does not select a layout: a 48-byte wire carries no ciphertext and no tag and could never open.
- `codec_id` + `layer` are **clear metadata, NOT inputs to the AEAD** (CC 5.3.3.2.1) — a fetching node asks for, and a serving peer sends, one layer by `(codec_id, layer)` without compromising the inner DEK; no relay reads it; tampering causes mis-decode or drop, never a crypto break.
- **One chunk, one link payload (normative).** A sealed A/V chunk rides the RNS Link whole, under the CC 5.3.3.2.5 outer seal (AES-256-GCM), itself inside the link's CC 5.3.3.5 E1 transit seal; it is never fragmented into symbols. The sender checks the chunk against the link's channel payload MDU before sending and refuses an oversize chunk as `ChunkTooLarge`, never splitting it. No symbol envelope, no XChaCha20-Poly1305 and no `K_symbol` (CC 5.4.2) is on this path.

5.3.3.2.4 `ChunkLayer` + `ReceiverLayerPolicy`: SVC layer model (normative)

`ChunkLayer` is the 3-byte SVC layer descriptor in the chunk header (`spatial`, `temporal`, `quality`, each `u8`). Each axis is **monotonic**: layer `0` is the base (lowest fidelity, always required); each increment is an additive enhancement. A receiver reconstructs from the prefix `0..=max_spatial` × `0..=max_temporal` × `0..=max_quality` of cells. The base cell `{0,0,0}` is the **"blinking dot"** — the minimum a participant can subscribe to. For `codec_id = 0xFF` (opaque) the layer MUST be `{0,0,0}`.

`ReceiverLayerPolicy { max_spatial, max_temporal, max_quality }` (each `u8`) is the per-receiver drop policy. It is **advertised over the existing `federation_session` / `key_grant` entitlement surface — NOT a new wire** — and the sender drops chunks above the cap without re-encoding. `admits(layer)` is the per-axis test `spatial` ≤ `max_spatial` ∧ `temporal` ≤ `max_temporal` ∧ `quality` ≤ `max_quality`; a chunk tagged `codec_id = 0xFF` MUST be

admitted **unconditionally** regardless of policy (the fan-out filter short-circuits before consulting `admits`). Canonical policies: `BLINKING_DOT = {0,0,0}`, `UNCAPPED = {255,255,255}`. This composes with the inner-once / outer-N fan-out optimization: the inner seal runs once per chunk; the outer seal runs only for the `(receiver, chunk)` pairs the policy admits.

5.3.3.2.5 Double-seal + deterministic nonce derivation (normative)

`double_sealed_ciphertext` is **outer-AEAD(inner-AEAD(chunk_plaintext))** — two independent AES-256-GCM layers (12-byte nonce + 16-byte tag, standard ring layout each). The **inner** seal is end-to-end (the epoch-DEK content seal); the **outer** seal is the per-RNS-Link transit wrap (a relay sees the outer layer only, never plaintext — the CC 5.3.3.5 E1 two-layer posture). Both nonces are **deterministic** (no nonce is transmitted — every holder recomputes; collision-safety rides the CC 5.3.3.2 single-sender-per-`(stream_id, epoch)` invariant).

One chunk seal (normative). The inner seal **is** the CC 5.3.3.1 chunk seal: the STREAM nonce `prefix[7] ‖ counter_be[4] ‖ last_flag[1]`, with `counter` = the chunk's `chunk_seq` (the header's `u64` field carries it; it MUST stay below 2^{32} within an epoch, the 5.3.3.1 ceiling, with the forced epoch roll before wrap) and `last_flag` set on the epoch's final chunk. There is no second inner nonce, so a live chunk is stored as it was, a stored chunk can be served into a call, and CC 5.3.3.4's reconnect-then-pull works for a call. Live and stored chunks are one object model, differing only by subscriber offset. The **outer** nonce is:

```
outer_nonce = SHA-256( b"CIRIS-AV-OUTER-V1" || link_id || link_seq_be8 )[0..12]
```

The label bytes (`b"CIRIS-AV-OUTER-V1"`, ASCII, no terminator) are a **domain separator pinned by this section**. `link_seq` is `u64` **big-endian** and monotonic per RNS Link (transit replay guard).

Where the two inputs come from (normative). Neither nonce input rides the frame. `last_flag` is set by the epoch terminator and is not a header field (CC 5.3.3.1). `link_seq` is a per-`(link, stream)` counter both ends keep and is never carried in the frame: it starts at 0 for the first frame on the link and advances by one per frame; the sender advances it once per frame sealed; the receiver advances it only on a frame that authenticates. After loss, the receiver counts the frames it dropped before its sink and reports that count on its next frame, and the sender's hop tries, in order, `next + dropped`, `next`, and a bounded slack above that — in this edition eight counters, at most ten opens per frame — and never a counter below `next`, so a replay never opens. Loss neither end observes is covered only by that slack; beyond it the hop ends loudly, and there is no in-band resync. **Conformance is proven by the vector set** for the outer nonce and `to_bytes`, published with CIRISVerify; the inner seal's vectors are 5.3.3.1's. `codec_id` and `ChunkLayer` stay clear metadata outside both AEADs.

5.3.3.3 Per-stream log + stream-root (normative)

A live stream is its own transparency log, which is what makes a producer accountable for what it streamed: the signed roots commit the producer to a single chunk history. For each live_stream:

- `log_id = "stream:<stream_id>"` — the bare `stream_id` is the stream's logical identity; the log identifier as signed and served is the string `stream:<stream_id>`; chunks = leaves; stream-root = `SignedTreeHead{ log_id: "stream:<stream_id>", tree_size: chunk_count, root_hash, timestamp, signature }`. **A live stream produces these at the cadence below whether or not anyone records it**.

- **Producer signs the STH — MANDATORY** authenticity root. The producer signs **the signed tree head's signing bytes**: no domain label; the fields in this order, every integer little-endian: `u32 len(log_id) || log_id (UTF-8) || u64 tree_size || root_hash[32] || i64 timestamp (seconds since the Unix epoch) || u32 timestamp (subsecond nanoseconds)`, where `log_id = "stream:<stream_id>"` and `root_hash` is the RFC 6962 Merkle root over the first `tree_size` leaves, each leaf the chunk's SHA-256 in `seq` order hashed under the `0x00` prefix and each inner node under the `0x01` prefix. An inline file is a stream of one leaf: `stream_id` is the inline blob's stream id and `tree_size` is 1. The signature is hybrid: Ed25519 over the signing bytes, then ML-DSA-65 over `signing_bytes || ed25519_sig`. The CC 5.3.1 `sha256("ciris.sth_cosign.v1\n" || ...)` digest is the **witness cosign input only**; the two preimages are distinct by construction and neither verifies as the other. The envelope-bearing wrapper around the head takes its canonical bytes from CC 2.6.1 JCS.
- **Witness cosign — OPTIONAL**, via the CC 5.3.1 path verbatim. This is the best-effort / accountable split:
  - **Best-effort** (open media) → producer-signed root only
  - **Accountable** (paid media, registry propagation, emergency) → witness cosign per CC 5.3.1.1 consistency-proof, which is the **anti-equivocation guarantee** — producer cannot show different chunk-K to different subscribers nor rewrite mid-stream
- **Cadence**: producer publishes a signed root every `K` chunks OR `T` seconds (whichever first), **always at an epoch boundary** (CC 5.1) + at `sealed_at`. Witness cosign runs **per-epoch** (coarser than per-K to keep cosign-quorum cost off the hot path). Default pins: **K=64, T=2s** (operator-tunable)
- **Incremental verify**: each leaf's "root-after-K" lets a subscriber verify chunk K's inclusion against the nearest signed STH $\geq$ K via the CC 5.3.1.1 consistency path. No new commitment structure beyond the per-leaf chain-link + periodic STH that CC 5.3.1 already provides.
- **Accountable-stream quorum**: Policy E (CC 4.4.3.1 locality-scaled quorum) applies — not a fixed N. Emergency-channel roots want a higher quorum than a paid-media stream; locality scaling provides the gradient.

5.3.3.4 Liveness invariant: entitled vs reachable (normative)

Two sets are NEVER conflated — the durable record of who is *allowed* in, and the ephemeral fact of who is *currently here*. Mixing them would turn presence into a logged, federated attestation, which the fail-secure posture forbids:

- **Entitlement roster** (Persist-owned): signed CEG envelope, Edge-propagated, durable, logged. It's **evidence** — it MUST propagate + be auditable. Per CC 4.4.3.2 Policy M community-membership composition
- **Live-reachability set** (Edge-owned): generalizes the holding claim's 72 h liveness (CC 3.1.3.3) down to seconds-to-minutes for live-multicast. Node-local presence tracker. **NEVER an attestation, never a holding claim, never replicated, never logged**

Fan-out invariant: `fan_out(C) = entitled(C)` $\cap$ `reachable(now)`.

Heartbeat-suppression discipline: this is a **producer-side-refusal invariant** (same class as the CC 5.2 no-holding-claim-leaves-a-cohort rule). Missed (entitled-but-unreachable) members fall back to pull on reconnect — substrate does NOT keep retrying push, does NOT emit a "delivery_failed" attestation, does NOT log liveness state. The reconnect-then-pull catch-up rides CC 5.1 `history_on_join`; under the one chunk seal (CC 5.3.3.2.5) the pulled chunks open with the same nonce as the pushed ones, so it works for a call.

5.3.3.5 Transport: Edge layer (normative)

Decision	Behavior
E2 — pull-only	Broadcast and large-group one-to-many delivery is **pull-only**: producer seals chunks under the epoch DEK → emits holding claims at the stream's scope → subscribers pull via the existing `ContentFetch` path. There is no relay tree; the small-group realtime profile of CC 5.3.3.2 pushes on direct links, and `fan_out` in E3 names the delivery set, never a relay topology.
E1 — two-layer crypto (security-critical)	Transit-key is a **hop-by-hop transport wrap UNDER the E2E epoch DEK** (two independent crypto layers); the whole rule is stated as **E1, in full**, directly below this table.
E3 — fan-out = entitled ∧ reachable	**Persist owns durable entitlement** (the roster: signed CEG envelopes, replicated, logged). **Edge owns transport-reachability** via a node-local presence tracker. Fan-out targets the intersection. Reachability is NEVER an attestation, never a holding claim, never replicated, never logged — consistent with the CC 5.2 `cohort_scope: self|family` structural-invisibility shape
E4 — durable side rides existing federation-attestation path	Durable entitlement (roster + epoch-key grants) rides the **existing federation-attestation Edge path** — just more `federation_attestations` rows. NO net-new Edge transport for the durable side. Net-new is only on CC 5.3.3.3 streaming-log endpoints

E1, in full. The two-layer rule, in numbered sentences:

1. Transit-key is a **hop-by-hop transport wrap UNDER the E2E epoch DEK** (two independent crypto layers). It MUST NOT replace the cascade — a relay never sees plaintext. *Relay* in E1 and throughout CC 5.3.3 means a transport node forwarding Link frames, never a media fan-out role (CC 5.3.3.2 has none); the member-relay tree of the Anonymous Tier (CC 5.4.6) is a different object with its own name. Transit-key is for path-confidentiality; epoch-DEK is for end-to-end content confidentiality.
2. **PQC in transit — the layered posture (normative).** The RNS link layer is classical (ephemeral X25519, Ed25519, AES-256-CBC + HMAC) and is **not relied on for post-quantum confidentiality**: the RNS link request is a fixed X25519‖Ed25519 layout, and a link-level ML-KEM would break interoperability with Python RNS.
3. The guarantee is carried above it, in three layers, all of them required: every content key is wrapped X25519 + ML-KEM-768 (the at-rest cascade, `wrap_algorithm: v2`, CC 5.1); every federation record is signed Ed25519 + ML-DSA-65 (CC 2.6.1.2); and **every byte an edge sends over an RNS link MUST be under a hybrid transit seal** keyed by an X25519 + ML-KEM-768 exchange bound to that link's `link_id` and to a transit pair **authenticated to the identity the initiator means to reach** (that identity's own hybrid-signed binding, a Rooted binding, or a pairing pin — never a bare or unsigned announce).

4. That seal is the CC 3.3.6.2 transit KEX, run once per link as its first frame and covering every frame that follows (replication, membership, pairing and the device-join request, chat row headers, the A/V outer seal alike), not only the A/V hop. On the pairing plane the initiator has no owner yet; the seal still binds its self-signed node key, never an anonymous one (CC 3.2, adding a device, (e)). The transit-key wrap E1 names is that seal.
5. **The pre-seal route fetch.** A dialler that holds no pair for the node it means to reach — two strangers on a bare mesh, each holding only the other's announce, which cannot carry a post-quantum pair — opens the link and sends, as its first frame, a **route request** naming the dialled key and saying whether it holds that key's record; the responder answers with its own hybrid-signed `transport_destination` row (and its own key record, when the dialler says it holds none) and nothing else, and only for its own key, from its own live row carrying a pair.
6. A stranger cannot check a row's hybrid signature without the signer's public record, which an announce does not carry; the key record is the same self-signed public row the key plane replicates to everyone, so sending it discloses nothing more.
7. The dialler checks that the signer and subject of each row equal the dialled key before it stores anything, admits the key record through the key door (proof of possession and the CC 2.6.8 fingerprint), admits the route row through authenticated admission, verifies it under the rule above (fingerprint, in-row separation, signature), and only then runs the handshake; a row signed by or about another key is refused and never stored.
8. The responder answers a route request at most once per link and rate-limits route requests node-wide before parsing them past their tag.
9. That public, self-authenticating row is the one thing besides the handshake that crosses a link unsealed: it is the row a relay would hand over and an announce already makes public, so the round trip discloses nothing an announce does not; the receiver order (per-sender, per-link and node-wide rate) runs before it, the responder serves nothing and buffers nothing until the seal completes, and a link whose first frame is anything else is dropped.
10. Where even that fails — no reachable responder — the pairing pin (CC 3.2 adding a device, (f)) is the failover, the only path for two nodes that cannot exchange a single public frame.
11. With that, there is no classical-only fallback for any content and no advisory phase: a node that does not run the seal cannot talk to the mesh until it upgrades; and there is no public classical lane — a standard LXMF client that cannot run the seal is outside the mesh and is not served by an edge's LXMF path. Classical-only transport is rejected for streaming.
12. The in-transit **KEX** identifier names a *different* construction from the at-rest `wrap_algorithm` and carries its own identifier string; the two MUST NOT be compared, substituted, or accepted in each other's field (CC 5.1 one-construction-one-identifier rule).
13. **Which key binds a non-member relay:** the relay's transit KEX pair published on its signed `transport_destination` binding (CC 3.3.6.2), static against the initiator's ephemeral; never a KeyPackage init key, never an ephemeral-only exchange; a node with no valid transit pair cannot relay.

5.3.3.6 Delivery receipts (normative)

A `delivery_receipt:{stream_id}` Contribution is a subscriber's signed acknowledgement that they received chunk K under the named stream + epoch. **Best-effort default**; opt-in for **accountable** profiles (registry propagation, emergency).

Canonical bytes (domain-separated + length-prefixed, in its own length-prefixed layout; per CC 2.6.1 the envelope-bearing wrapper is JCS-encoded, but the receipt's signed-bytes inner payload follows the explicit-length-prefix shape below). The two encodings live at different layers: the JCS bytes are what the `scores` row's envelope signature covers, as for any row and as for the stream root of CC 5.3.3.3; the length-prefixed bytes below are the receipt payload the subscriber signs inside that row:

```
receipt_signing_bytes =
    "ciris-delivery-receipt/v1"                      // domain separator: 25 ASCII bytes, no length
     prefix
  || u32_le(len(subscriber_key_id)) || subscriber_key_id   // UTF-8
  || u32_le(len(stream_id))         || stream_id           // UTF-8
  || epoch        (u64 LE)
  || chunk_root   ([u8; 32])
  || K            (u64 LE)
```

Little-endian, in this layout alone (normative). The receipt's two length prefixes, `epoch` and `K` are little-endian. This is one of the two little-endian layouts in this Part, the other being the stream-root signing bytes of CC 5.3.3.3: each is a signature preimage, so its byte order is fixed as written, and a change would break every signature over it. Every other integer in this Part is big-endian, and a producer MUST NOT carry either layout's byte order into any other construction.

Order and lengths (normative). The signed order is the order written: `subscriber_key_id` precedes `stream_id`, even where a struct declares `stream_id` first. Each length-prefixed field MUST fit a `u32`; a longer field is refused, never clamped. `received_at` is not signed: it is the receiving node's local note and no part of the preimage.

Both `epoch` (key-rotation index — for per-epoch entitlement / billing) and `K`/`chunk_root` (chunk position + its committed root) are independent indices — both required (each names a distinct authorization scope).

Verify check is a JOIN, NOT a sig-check:

1. **Signature valid** over the canonical bytes (subscriber hybrid Ed25519 + ML-DSA-65 sig, ML-DSA-65 over the bytes ‖ `ed25519_sig`, the bound form). Necessary but **not sufficient**.
2. **`chunk_root` is a real published STH root** — MUST equal a `SignedTreeHead.root_hash` actually published for `log_id = "stream:<stream_id>"` (CC 5.3.3.3) at `tree_size` $\geq$ `K`. A phantom / self-invented root → REJECT. For accountable streams, "published" means **witness-cosigned** (the CC 5.3.1 path), so the subscriber cannot collude with the producer on a private root.
3. *(Recommended for accountable)* **Inclusion proof** chunk K → `chunk_root`. Upgrades the receipt from "subscriber saw a root" to "subscriber saw a root that provably commits to chunk K".

Semantics — proof-of-DELIVERY, not proof-of-CONSUMPTION: the receipt attests that the subscriber signed over a published root at `tree_size` $\geq$ `K` and, with the inclusion proof, that the root commits to chunk K. It does NOT prove they decrypted those bytes (they may not hold the epoch DEK). Consumers MUST NOT overclaim a delivery receipt as proof of consumption. Per the fail-honest invariant + CC 1.7 "Verify authenticates origin, does not compose 'delivered'/'owes N'", Verify's role is validation-not-adjudication: emit the validated receipt as an attestation on `delivery_receipt:{stream_id}` — the "delivered" verdict is consumer policy.

Accountable-stream receipt quorum: Policy E (CC 4.4.3.1 locality-scaled) — same shape as the CC 5.3.3.3 STH quorum.

5.3.3.7 Framing (normative)

A stream is **its own per-stream transparency-log instance** (`log_id = "stream:<stream_id>"`, CC 5.3.3.3). A `live_stream` MUST NOT append chunks into the federation provenance log (CC 5.3.1's global log carrying builds / licenses / identities) — millions of media chunks would pollute provenance and inflate the global tree. The CC 5.3.3 path **reuses** the CC 5.3.1 `SignedTreeHead` / `ConsistencyProof` / `WitnessConsistencyProof` / cosign abstractions, instantiated per-stream as separate log instances under the same RFC 6962 algorithm.

The 1+4 wire-format lockdown holds: there are no new `attestation_type` values. Stream chunks ride content addressing; stream-roots ride the existing `SignedTreeHead` shape; delivery receipts ride `scores` against the new `delivery_receipt:{stream_id}` reserved prefix (CC 3.4.6).

5.3.3.8 What the streaming surface documents

Scope pointer: the CC 5.3.3 streaming surface — delivery axis (CC 5.3.3.3–CC 5.3.3.4) — does **not** change the 1+4 primitive set (CC 2.4) (delivery rides existing primitives), does **not** bundle the streaming-half substrate implementation, keeps multicast relay/fan-out pull-only, and leaves the K / T / MAX_CHUNKS_PER_EPOCH constants + accountable-stream quorum operator-tunable.

5.3.4 Trust-root discovery: the GenesisBundle + accord holders

These endpoints publish the trust root itself — the accord-authorized GenesisBundle, and the accord holders it names — so a verifier can discover *whom* to trust before trusting anything. A consumer MUST verify the bundle's quorum against its own records before promoting any field to a trust root: this is the fidelity / fail-secure floor for the whole discovery surface. Nothing a serving install signs on its own account is a root.

`GET /v1/trust-root/bundle` — the portable trust root (`/v1/steward-key` is an alias)

Returns the **GenesisBundle**: the charter, the accord-holder roster, the scopes and the serve-node grants, hybrid-authorized by the accord holders at the genesis ceremony. A registry publishing its own key as its own root, `hardware_class` self-attested, is the weakest form of a producer claim (CC 4.2.2.1); the registry is a conferred slice of a canonical node, not a root. The same body is served at both paths.

Response (`200 OK`):

```
{
  "bundle": {
    "version": 4,
    "family_key_id": "humanity-accord",
    "holders": [ "<SignedKeyRecord>", "..." ],
    "serve_nodes": [ "<SignedKeyRecord ciris-canonical-1-...>" ],
    "serve_transport_destinations": [ "<SignedAttestation transport_destination of ciris-canonical-1-...,
     transit pair inside>" ],
    "consensus_protocol": "majority",
    "attestations": [ "<SignedAttestation: charter(s), lifecycle, grants, the ciris-canonical birth record
     >" ],
    "authorizations": [ "<GenesisAuthorization A1>", "<GenesisAuthorization B1>", "..." ],
    "produced_at": "<rfc3339_canonical>"
  },
  "bundle_fingerprint": "sha256:<64-char-lowercase over the JCS-canonical bundle>",
  "charter_root_key_id": "<read off the bundle; verify it>",
```

```
  "served_by": { "node_key_id": "<relaying node>", "accepts_this_root": true }
}
```

Authority lives inside `bundle` and nowhere else (normative). The bundle is **tamper-evident, not authentic** (CC 3.2 T5): its holders' hybrid authorizations over the charter prove the bundle is internally consistent, and an attacker's self-consistent bundle under the same identifiers verifies identically. A consumer therefore MUST hold an **out-of-band anchor before accepting a bootstrap bundle** — either the pinned `bundle_fingerprint` (the `sha256:` over the JCS-canonical bundle, published with the release and baked into conformant builds) or the pinned fingerprints of the accord-holder keys — and MUST compare the received bundle against it at attach time; `ciris-canonical` and `humanity-accord` are **names, not anchors**, and a consumer with no anchor MUST NOT promote anything in the response to a trust root. With the anchor held, the consumer re-derives the quorum from its *own* records (`verify_bundle_quorum` — the authorizations counted against the roster the consumer holds or has just pinned), and from then on the root is valid until revoked (T4). Everything outside `bundle` — the served fingerprint, the charter root id, `served_by` — is unsigned convenience metadata the consumer recomputes (the fingerprint against its pin) or ignores; the served `bundle_fingerprint` is never itself the anchor. There is deliberately **no `response_signature`**: signing the wrapper would prove only that the relaying node said it, which is worthless as a root, and it would invite consumers to check the envelope instead of the artifact. `hardware_class`, `signature_mode` and `threshold_policy` MUST NOT appear on the outer envelope; a holder's custody class rides its signed key record and is corroborated, or not, per CC 4.2.2.1. `served_by.accepts_this_root: false` is a legitimate state — a node may relay a root it has not accepted (CC 3.2 T3) — and is the operator's un-trust lever made visible.

A consumer pins the community anchor `community_key_id: ciris-canonical` and the accord family `humanity-accord`, resolves the live member set via `resolve_community` (CC 4.4.3.2.4), and never hard-pins a serving install's fingerprint. A root is valid until revoked, never until a timer lapses (CC 3.2 T4); what freshness gates is *attaching*, under T4a, and the object that carries it is served beside the bundle.

The lineage head beside the bundle. The genesis head of each lineage — the `humanity-accord` family record and the `ciris-canonical` community birth record (CC 3.2 T6: the signed roster record at a version is the head) — and the charter that carries each root's `attach_window_secs`, `witness_cadence_secs`, `witness_quorum` and `witnesses[]`, are members of `bundle.attestations` and are therefore pinned by `bundle_fingerprint` with everything else in it. The route serves, outside `bundle`, a `lineage_head` member: the **head chain since genesis** of each lineage — every version of the signed roster record, the charter bytes each names, and the cosignatures collected over each under CC 5.3.1's lineage-head domain — and the same chain is served at `GET /v1/trust-root/lineage-head` (with `?since=<digest>` for a consumer refreshing before it attaches), any object in it fetchable by digest. It is unsigned convenience like everything outside `bundle`: the consumer walks `prev_head_digest` from the genesis head it holds from the bundle, checks each head's quorum against its **parent's** fold and each cosignature against its **parent's** directory (never the head's own `charter_digest`, which a fork head chooses), counts only independent witnesses, and applies CC 3.2 T4a — fewer than `witness_quorum` is `trust_root_head_unwitnessed`, a newest counted cosignature older than `attach_window_secs` is `trust_root_head_stale`, and in either case the consumer fetches a fresher head or refuses; it never attaches on the stale one. A consumer SHOULD obtain at least one counted cosignature from that witness directly. **Charter defaults.** With witnessed mode off (`witness_quorum: 0`, no directory; CC 3.2 T6) a root attaches by the pinned `bundle_fingerprint`, with `attach_window_secs` **604800**. The accord's charter car-

ries `halt_fuse_secs` **86400** (24 hours, CC 4.2.1.1). With witnessed mode on, turned on by an amendment signed by a strict majority of the standing accord roster, the defaults are, for both `ciris-canonical` and `humanity-accord`: `attach_window_secs` **604800** (7 days), `witness_cadence_secs` **86400** (24 hours), a directory of **exactly three** independent witnesses and therefore `witness_quorum` **2** ($\lfloor 3/2 \rfloor + 1$ — a fourth witness raises it to 3, since two of four is not a strict majority); all four are charter members, changed only by re-scrub, and the window exceeds the cadence plus skew as T4a requires. Once attached, T4 governs: the served head's age is a liveness signal on the trust surface, never an expiry.

The bundle is the Rooted seed for transit pairs (normative; `GENESIS_BUNDLE_VERSION` 4). Under the per-link hybrid seal of CC 5.3.3.5 E1 a node dials only a peer whose transit pair it holds **authenticated to the identity it means to reach** — that identity's own hybrid-signed `transport_destination` row (CC 3.3.6.2), from any source; or a Rooted binding; or a pairing pin (CC 3.2 (f)) — and such a row otherwise arrives over a sealed link, so a device that knows no one needs a first row from somewhere, and for the seated serve nodes it is this bundle. **A canonical node is one source of route rows, never a required party** (the premise of this document is a network with no company in the middle, and a trust decides what a node serves, CC 3.2 T8, not whether two keys may speak privately): two peers holding each other's signed route rows — from a contact code, from the owner's own hand at first contact (the contact rule of CC 5.4.6), or, for an unowned node, from any relay of the bootstrap kinds — seal, exchange the allegiance facts and root each other directly. The bundle carries, in the member **`serve_transport_destinations`**, the seated serve nodes' signed `transport_destination` rows (CC 3.3.6.2), each carrying its transit pair; the member is inside the `authorization_digest` preimage, so the holders' quorum authorizes the exact set as it authorizes `serve_nodes`. At bundle verification each row MUST be signed by a serve node listed in the bundle's `serve_nodes` and MUST carry a pair that passes the in-row separation; any failure fails the bundle closed. At first open the rows are seeded through the ordinary transport-destination admission and land Rooted, so a fresh node with no network holds a canonical node's pair, dials it first under the seal, and replication brings it the mesh's announced bindings. A bundle re-mint comes in two forms. A **fresh genesis** is the final-genesis ceremony: every holder signs the charter and then the genesis records and the authorization, because only each holder can give their own recovery commitment and successor seat and the founders sign the `ciris-canonical` birth — unanimity is what that ceremony gathers, and any change to the charter or the roster takes it. An **amend** carries the pinned, verified bundle's charter, grants, lifecycle row, serve-node records, both genesis records and `produced_at` forward **byte for byte** — every carried signature stays valid over its own unchanged envelope — adds `serve_transport_destinations`, and signs only the new `authorization_digest`; it finishes at **M distinct holder authorizations** (the charter's `quorum:2/3`, CC 4.2.3; verification requires `max(M, strict majority of the roster)`, which is 2 of 3) and refuses if any carried byte differs from the pinned input, so an amend cannot smuggle a changed charter or grant. Seeding the canonicals' transit pairs is the same class of act as blessing a canonical, already a strict-majority act, so two holders gain no power they lack. The rows the bundle carries are **signed** `transport_destination` rows, each the node's own signed self-emission. **The first seed is emitted offline:** under the seal no existing node can dial a canonical whose pair it does not hold, and the re-mint cannot read a row the canonical has not emitted, so the cycle is broken on the host — a seated canonical's operator emits the node's hybrid-signed row, pair included, from the node's own custody with no network (the transit pair is derived deterministically from the custody seed, so the offline row and the row the node emits live later carry the same pair); the ceremony re-mints from those rows. Bundle verification checks each row's signature and its signer's seat, never its age (CC 3.2 T4: valid until revoked), so a row emitted before the ceremony seeds a node opened long after; the node's live row supersedes it on the first sealed link, same pair,

later `asserted_at`. **Overlap rule:** a seated serve node MUST keep the pair the pinned bundle carries valid until a re-minted bundle carrying its new pair has propagated, or make the rotation itself a re-mint; the node's own rotation door enforces it — a rotation that would retire the pinned pair before its replacement is Rooted on every node is refused. **A rotation is a quorum act, not a re-mint:** the holders' quorum re-binds a canonical's destination and transit pair by the accord-signed address update of CC 3.3.6.2, which lands as a Rooted `transport_destination` superseding the bundle-seeded pair on every node that holds the root; the re-mint is needed only to change what a *fresh* node seeds, and a fresh node that seeds the old pair follows the supersede chain to the new one once it has sealed to any canonical. The bundle is the only seed before the first sealed link; two other ways a transit pair reaches a node before any sealed link are defined: the owner's pairing payload at pairing (CC 3.2, adding a device, (f)), pinned for that link alone, and the pre-seal route fetch of CC 5.3.3.5 step 5, by which a dialler holding only a stranger's announce receives that key's own hybrid-signed route row as the link's first frame.

What crosses before Rooted (normative). A trust decides what a node *serves*, never whether two keys may speak privately. On a sealed link to a peer the node has not Rooted, what the peer's own standing admits still crosses: the bootstrap kinds and the allegiance facts, the pairing plane, rows addressed to the peer, and every row a bounded cohort delivers to that peer as its member under CC 5.2 — `self`, `family`, `community`, `affiliations` — because cohort delivery is consent and membership, not service. What waits for `Rooted` (CC 3.2 T8) is what the node serves to a third party: a federation-tier row, and any self-authored row about someone other than the peer.

`GET /v1/accord-holders`

Three named holders with hybrid pubkeys + per-holder `hardware_class` + `provisioned` flag. A holder not yet provisioned carries a placeholder fingerprint and `provisioned: false`; consumers MUST NOT honor `constitutional` invocations against placeholders. The holders' authority is the bundle above — this route is a projection of `bundle.holders`, and a consumer cross-checks it against the bundle rather than trusting the projection.

`GET /v1/accord/holders`

UI wrapper around `/v1/accord-holders` with per-holder `accord_emissions[]` for UI rendering. The same projection rule: **no response signature** — the wrapper carries no authority, a consumer cross-checks every holder it shows against `bundle.holders` under the anchor it already holds, and `accord_emissions[]` are the holders' own signed rows, verified individually.

`GET /v1/rotation-history`

Chronological rotation events of the Registry's signing keys.

5.3.5 Other Registry endpoints

`GET /v1/builds/{version}` returns the BuildRecordResponse with a `federation_provenance` block. `GET /v1/verify/build-manifest/{project}/{version}/{target}` returns the verbatim signed BuildManifest. `GET /v1/agent_files/{kind}?platform_or_target=...` returns the CC 4.4.3.7 trust-composition layers. `GET /v1/partner/{key_id}` composes ProfileScorecard data from existing tables.

Full response schemas for these endpoints are published as a versioned OpenAPI document alongside this one.

5.3.6 Common response shape

All CEG endpoints return:

- **Content-Type**: `application/json` (`Accept: application/json` honored; other types respond `406 Not Acceptable`)
- **CEG-API-Version header**: `CEG-Version: <current spec major.minor>` on every response (this document's version); clients SHOULD echo `CEG-Accept-Version: <pinned-version>` on request, naming the version they were built against. Per CC 2.6.4 SemVer policy, MAJOR mismatch is a wire-incompat reject; MINOR mismatch is compatible (clients MAY warn).
- **Time-Source header**: `X-CEG-Server-Time: <rfc3339_canonical>` per CC 2.6.2 for client clock-skew bounds
- **Pagination** (where applicable): `?cursor=` + `?limit=` query params; response includes `next_cursor` (null if exhausted) and `total_estimate` (server's best estimate, may be approximate)

5.3.6.1 Error envelope

All error responses MUST conform to:

```
{
  "error": {
    "code": "<ENUM_VALUE>",
    "http_status": <int>,
    "message": "<human-readable>",
    "request_id": "<server-assigned>",
    "details": {<error-specific fields>}
  }
}
```

HTTP status	Error code	Meaning
400	`MALFORMED_REQUEST`	Invalid JSON, missing required field, bad field type
400	`CANONICAL_BYTES_VIOLATION`	Date-time / hex / encoding doesn't match CC 2.6.2 / CC 2.6.3
401	`UNAUTHENTICATED`	Bearer token missing or invalid
403	`RESERVED_PREFIX_VIOLATION`	Producer attempted to emit under a reserved prefix without authority per CC 3.4
404	`UNKNOWN_WITNESS`	Witness key_id not registered in directory (CC 5.3.1)
404	`NOT_FOUND`	Generic resource not found (build, partner, key)
409	`IDEMPOTENT_CONFLICT`	Replay detected (e.g., duplicate `(tree_size, witness_key_id)` cosignature with different signatures)
422	`SIGNATURE_VERIFICATION_FAILED`	Ed25519 or ML-DSA-65 failed to verify; `details.algorithm` names which
422	`CLOCK_SKEW_VIOLATION`	`signed_at` exceeds CC 2.6.7 ±5 minute tolerance

HTTP status	Error code	Meaning
422	`WITNESS_QUORUM_NOT_MET`	Insufficient cosignatures to validate
422	`CONSISTENCY_PROOF_INVALID`	A witness cosignature's CC 5.3.1.1 consistency proof against the prior STH it cosigned is absent or does not verify. A missing proof when a prior STH exists, or a tree_size behind the witness's prior cosigned STH, is `MALFORMED_REQUEST` instead.
422	`LINEAGE_HEAD_INCONSISTENT`	A lineage-head cosignature (CC 5.3.1) names a prior head the Registry did not record for that witness, or a head that neither equals nor descends by `prev_head_digest` from it. A quorum-signed head that does not descend from the lineage's *witnessed* head is not this error: it is recorded uncosigned as `hard_case: lineage_fork_attempt` evidence (CC 3.2 T6)
413	`ENVELOPE_TOO_LARGE`	Canonical JCS bytes exceed the CC 2.6.1.3 1 MiB bound. Raised at **every** write path (HTTP body, capsule, FFI, tier-ingest), not only at an HTTP gate — the CC 2.6.1.3 rule is an admission bound, not a transport limit. The storage-layer spelling `federation_envelope_too_large` maps to this wire code; the wire code is the conformance surface.
429	`RATE_LIMITED`	`X-RateLimit-*` headers set; `Retry-After` honored
500	`INTERNAL_ERROR`	Server-side fault; request_id usable for support
503	`WITNESS_DIRECTORY_UNAVAILABLE`	Substrate replication lag exceeds liveness bound

Admission refusal tokens — `attestation_type_unregistered`, `federation_malformed_row` with its closed reason set, `federation_set_semantics_unsorted` (CC 2.6.1.2), `asserted_at_stale` (CC 2.6.7), `trust_root_head_unwitnessed` and `trust_root_head_stale` (CC 5.3.4), and the census tokens of CC 2.1 — are refusal reasons carried in the row's refusal, not HTTP statuses, and this table does not map them.

Rate-limit headers on every response: `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` (seconds-until-reset epoch).

5.4 Scope-Native Privacy wire profile

The wire-format absorption of the Scope-Native Privacy construction. These are the deterministic, byte-pinned primitives a conformant substrate uses to make real and cover traffic indistinguishable below the cohort boundary — record addressing, per-symbol AEAD, uniform fragmentation, the MLS Welcome wrap, the witness-chain discipline, and announce suppression. They are additive to the wire layer; the 1+4 surface is unchanged.

5.4.1 Deterministic `record_id` (CBOR profile, normative)

Scope-native distribution addresses each record by an HMAC-opaque `record_id` rather than any plaintext key. The `record_id` is a deterministic, group-keyed commitment to the record's identity: every member of the MLS group derives the identical `record_id` for a given record, while non-members observe an unlinkable 32-byte tag. This is the addressing argument of the deterministic ALM "directory" — a pure function, never a queryable authority (CC 4.4.3.2.1 cohort-scope lattice).

The preimage `record_id_input` is the Core Deterministic Encoding of a fixed four-entry map; `record_id` is its HMAC under the per-epoch key `K_record_id`. The preimage, the key derivation, and the resulting tag MUST be byte-identical across producer, substrate, and every consumer — any disagreement on CBOR byte order, integer minimal-encoding, major-type selection, or HKDF/HMAC suite yields a different `record_id`, silently partitioning holders of the same record. This is the same byte-exactness hazard class the STREAM nonce derivation guards (CC 5.3.3.1).

Per-epoch key. `K_record_id` is derived in two stages: an RFC 9420 exporter call under the record label, then a bare HKDF-Expand (no Extract, no MLS KDF-label framing) over that exporter output. `K_symbol` (CC 5.4.2) is the second stage of the same chain under its own label:

```
S_record    = MLS-Exporter(label = "ciris-scope-record-v1", context = "", length = 32)

K_record_id = HKDF-SHA256-Expand(
    PRK  = S_record,
    info = "ciris-edge/scope-privacy/record-id/v1"  (ASCII),
    L    = 32
)
K_symbol    = HKDF-SHA256-Expand(
    PRK  = S_record,
    info = "ciris-edge/scope-privacy/symbol/v1"     (ASCII),
    L    = 32
)
```

The HKDF hash here is **SHA-256** (the ciphersuite 0x004D key-schedule hash). Implementations MUST derive `S_record` by the exporter call under `ciris-scope-record-v1`, and `K_record_id` and `K_symbol` by the bare HKDF-Expand over `S_record`; the second-stage labels MUST NOT be routed through RFC 9420 `MLS_Exporter()`, and the raw `exporter_secret` is never used directly. `S_dest` (CC 5.4.6) is a separate exporter under its own label.

Preimage and commitment. Each record:

```
record_id_input = CBOR_dCE({
    "v":   1,
    "epc": mls_group_epoch,
    "iid": internal_id,
    "typ": record_type,    // integer per RecordType table below
})
record_id       = HMAC-SHA3-256(K_record_id, record_id_input)
```

`CBOR_dCE` is RFC 8949, section 4.2.1, Core Deterministic Encoding (shortest-form integer encoding, definite lengths only, no floating-point unless required), with map keys ordered **by encoded-length first, then lexicographically by encoded key bytes**. The four-entry map at `v=1` therefore serializes keys in the canonical order `v, epc, iid, typ`.

Major types are pinned. Within the map the four keys (`v`, `epc`, `iid`, `typ`) are CBOR text strings (major type 3). The values of `v`, `epc`, and `typ` are unsigned integers (major type 0). The value of `iid` is a **CBOR byte string (major type 2)** — `internal_id` is committed as its raw bytes, never as a text string, even when those bytes are human-readable (e.g. `record-0001`). Encoding `iid` as a text string (major type 3) produces a different preimage and a different `record_id`; the byte-string header is why the vectors below carry `0x4b`/`0x41` (major-2) and not `0x6b`/`0x61` (major-3) at the `iid` value position.

`RecordType` integer encoding (pinned for cross-impl conformance):

Type	int
reserved	0
SelfRecord	1
FamilyRecord	2
CommunityRecord	3
FederationRecord	4

Additional record types extend this table by amendment under CC 2.6.4.

Conformance vectors (normative). With `K_record_id = 0x11` repeated 32 times, conforming implementations MUST reproduce these exact CBOR preimage bytes and `record_id` tags (hex):

```
// Vector 1 -- CommunityRecord (typ=3), epc=7, iid="record-0001"
record_id_input = a461760163657063076369696 44b7265636f72642d303030316374797003
record_id       = 5428ddb514a8f8692cc4f254f3550ea75790f5069673e42afb6ef318517a0b21

// Vector 2 -- FederationRecord (typ=4), epc=300, iid="record-0002"
//   (forces 0x19 u16 epc header: bytes[8..11] == 0x19 0x01 0x2c)
record_id_input = a461760163657063190 12c636969644b7265636f72642d303030326374797004
record_id       = 04eebeee4d5b83f2fdd0012a205781e6c05fe9a587377e6161b347629a189ff2

// Vector 3 -- SelfRecord (typ=1), epc=16909060 (0x01020304), iid="x"
//   (forces 0x1a u32 epc header: bytes[8..13] == 0x1a 0x01 0x02 0x03 0x04)
record_id_input = a461760163657063 1a01020304636969644 1786374797001
record_id       = 79bee8b3f1e815a1df03ca9d83427dc5ab474e184f34e3876d3ef3c36559d6a3
```

The deterministic CBOR profile is normative here; the vectors are published with CIRISVerify.

5.4.2 Uniform symbol AEAD framing for scope-privacy symbols (normative)

Within both encrypted crypto tiers — the self/family **InvisibleEncrypted** tier and the community/affiliations **CommunityDek** tier (CC 4.4.3.2.1, resolved through `crypto_tier(cohort_scope, cohort_subkind)` with non-`infrastructure` subkind) — stored content moves and rests as encrypted chunks, replicated per item to its holders and never split by any node; on the scope-privacy plane each such chunk crosses a shaped link as symbols — the indexed fragments of CC 5.4.3, real and cover alike — and each emitted symbol is sealed under its **own** AEAD key, so a relay or holder lacking the keys holds indistinguishable-from-random ciphertext. The realtime A/V plane (CC 5.3.3.2) is not on this path: a sealed A/V chunk rides its link whole under the E1 seal (CC 5.3.3.2.3) and emits no symbol. The two tiers differ in the shape of their roster, not in discoverability to outsiders, which is none for both: self/family and community/affiliations content alike emit holding claims only within the cohort (CC 5.2 structural invisibility), so outsiders cannot discover the bytes exist; a community's roster may be larger, or open (CC 3.2 `public`), a family's is intimate, and the holder-inspectability principle (CC 4.4.3.2.1) says members hold what they can open and strangers hold nothing (keep/evict decision — but the bytes they hold remain sealed and unreadable: a chunk under its own seal, a symbol under the symbol AEAD below without `K_symbol` and `record_id`). The substrate MUST seal every symbol it emits with the framing below; a holder lacking both `K_symbol` and `record_id` MUST be unable to distinguish a real symbol envelope from a cover symbol envelope.

`record_id` is the per-record HMAC-opaque addressing tag of the group-keyed addressing construction (CC 5.4.1) — `record_id = HMAC-SHA3-256(K_record_id, CBOR_dCE({v, epc, iid, typ}))`. `K_symbol` and `K_record_id` are the per-scope subkeys of the one derivation chain of

CC 5.4.1 (`HKDF-SHA256-Expand` over `S_record`, the `ciris-scope-record-v1` exporter output, each under its own label); both rebind on MLS Add/Remove.

For each symbol the wire framing is byte-exact:

```
For each symbol:

    symbol_key = HKDF-SHA3-256(salt=record_id, ikm=K_symbol,
                               info="ciris-edge/scope-privacy/symbol/v1"
                                    || u16_be(symbol_index), len=32)
    symbol_envelope = (record_id, u16_be(symbol_index), nonce,
                       XChaCha20-Poly1305(symbol_key, nonce, fragment), tag)
```

Normative constraints:

- **`symbol_key` derivation is byte-identical across every implementation.** `salt = record_id` (the 32-byte HMAC tag), `ikm = K_symbol` (the 32-byte exporter subkey), and `info` is the **exact** concatenation of the UTF-8 bytes of the ASCII label `ciris-edge/scope-privacy/symbol/v1` followed by `u16_be(symbol_index)` (the symbol index as a 2-byte big-endian unsigned integer), with `len = 32`. The same ASCII label is both the second-stage `info` that derives `K_symbol` from `S_record` (CC 5.4.1) and this info-prefix; producer, substrate, and every consumer MUST recompute this exact value. Any BE/LE disagreement on `symbol_index`, or any drift in the label bytes, yields a different `symbol_key` → AEAD authentication failure (silent decryption failure).
- **The AEAD is XChaCha20-Poly1305 with a 24-byte `nonce` and the Poly1305 `tag`** carried in the envelope. The same suite seals **both** real fragments and cover fragments — there is no separate cover cipher, no cleartext cover marker, and no length distinguisher. A holder lacking `K_symbol` and `record_id` holds indistinguishable-from-random bytes (least-authority, under the standard assumptions on XChaCha20-Poly1305 and HKDF-SHA3 extract-then-expand); cover and real envelopes are observationally identical.
- **The fragment-set rebinds on MLS Add/Remove.** Because `symbol_key` chains from `K_symbol` and `record_id`, both of which rotate with the group epoch, a removed member's symbol keys do not open post-epoch fragments. Past-epoch `K_record_id` / `K_symbol` are deleted by honest holders per the community's archive policy.
- **`symbol_envelope` field order is fixed: `(record_id, u16_be(symbol_index), nonce, ciphertext, tag)`.** The cleartext `record_id` and `symbol_index` are the addressing argument to the HMAC-opaque directory function (the CC 6.1.6 deterministic ALM topology) and are NOT authority-bearing — they reveal nothing to a non-member, who cannot derive the corresponding `symbol_key`.

This framing is the byte-level analogue of the CC 5.3.3.1 per-chunk stream seal: the costly key agreement is amortized into the MLS exporter, and the hot path is a per-symbol symmetric AEAD with a deterministically derived key. It composes with — and never substitutes for — the at-rest DEK cascade (CC 4.4.3.4.1); the symbol AEAD protects the scope-privacy plane's fragments in transit and at any relay that forwards them, the DEK cascade protects the record at rest.

5.4.3 Uniform-envelope fragmentation & reassembly (normative)

A claim's bytes rarely fit one packet, and the split is where privacy is most easily lost: a naive fragmenter leaks payload size, count, and cadence to anyone on the path. The discipline here is

that every fragment on the wire — a real publication fragment or a synthetic cover envelope — is the **same size** and the **same AEAD shape**, so an on-path observer cannot distinguish a large publication from idle substrate maintenance. This composes with the chunk-seal of CC 5.3.3.1 (a *different* axis: streaming media chunks) — fragmentation is the substrate-emission layer beneath every send on a shaped link, not the streaming codec.

Fixed frame size on a shaped link (normative). On a shaped link (below) every frame is one constant size, the link's channel payload MDU, which differs by interface (about 400 bytes at the Reticulum base MTU, about 16 KB over TCP); a larger payload is fragmented to that size and a smaller one padded to it. Within a link the size is not a tuning knob, because a variable frame re-exposes the size channel that shaping exists to close. An unshaped link carries no padding requirement. There is no fixed envelope size for all outbound traffic: uniform frames add privacy only where both ends shape.

Link shaping (normative). Shaping hides when and how much real traffic a link carries by sending a constant stream of identical frames, real or cover.

1. **A link property, negotiated per link.** Shaping is agreed on each link immediately after the CC 5.3.3.5 E1 handshake, under its keys, not set per node. Each end offers a budget class, `server`, `home`, `phone-wifi`, `phone-cellular` or `off`; the link runs at the lower of the two offers, and a node divides a node-wide cap across its shaped links. Shaping is **off by default** until the steward sets a fleet default.
2. **Both directions or neither.** On a shaped link both directions are shaped at the negotiated rate: a responder that accepts a shaped link shapes its replies on it, and a node that cannot meet an offer declines the link rather than run it half-shaped. One shaped side beside one unshaped side is exposed by the unshaped side's timed replies.
3. **The stream.** Every frame on a shaped link is a constant-size sealed frame; real frames are fragmented to that size, and cover frames are indistinguishable from them under the seal. Frames leave on a per-link Poisson timer at the negotiated rate λ. Cover dominates by the budget rule: real traffic is at most a fraction ρ of the stream over a window W, so a link carries at most `target_rate = ceil(`$\rho \cdot \lambda \cdot$`W)` real frames per window, counted locally at each end; this real-frame budget is the one rate quantity this Part names. ρ and W are explicit settings, because they trade latency against how much load an observer can infer; the slow tail of delivery comes from W, not from the timer. Their defaults are in the table below.
4. **Scope.** Maintenance traffic (anti-entropy, replication, witness, repair) and ordinary rows are shaped. The real-time audio and video plane (CC 5.3.3.2) is never shaped.
5. **What shaping does not hide.** Link setup stays observable: the request to open a link to a derived destination is visible, and shaping runs only inside established links. A node with any unshaped link can be told apart from a node that shapes every link. Shaping hides timing and volume inside a shaped link, not which links exist (CC 5.4.6, CC 1.13.3.1).

Fleet defaults (normative defaults; every value stays a per-node setting). Shaping is **off** by default. When a node turns it on, it offers its budget class with these values:

Budget class	ρ	W	Byte budget per link
`server`	0.5	600 s	7,000 B/s
`home`	0.5	600 s	2,800 B/s
`phone-wifi`	0.5	600 s	1,400 B/s

Budget class	ρ	W	Byte budget per link
`phone-cellular`	0.5	3,600 s	350 B/s

A node's shaped links together use at most eight links' worth of its class budget.

Why these values (informative). They come from a model run on the production scheduler and budget meter over a virtual clock with real row sizes. They are not measurements from a live mesh, and they are revised when live measurements exist. What the model found:

- ρ must sit above the link's real load with margin. When real load exceeds ρ the link saturates: useful throughput is capped at $\rho \cdot \lambda$ and the sender is held back. $\rho = 0.25$ halves useful throughput and saturates at ordinary load. $\rho = 0.8$ leaves the peer almost no uncertainty about the real share. 0.5 is the balance.
- Well below ρ, neither ρ nor W changes latency; the Poisson timer sets it (about 1.6 s mean on `server`, 4 s on `home`, 8 s on `phone-wifi`, 26–32 s on `phone-cellular`). Near ρ, W decides: a 60 s window costs one to two minutes at the mean and up to five at the 95th percentile, 600 s costs 6 to 36 s, and 3,600 s returns to the timer's base.
- `phone-cellular` uses W = 3,600 s because at its low rate a 600 s window admits only 75 to 260 real frames, and near the limit delay rose to about 380 s mean and 910 s at the 95th percentile, against 29 s and 54 s at 3,600 s. No phone class uses W = 60 s.
- The cover share is the unused capacity (90 % at 10 % load, 50 % at 50 %), the same at every W.
- An observer of the wire sees, on a shaped link at any setting, that the link exists, its endpoints, and a constant-size frame stream at the rate λ; it does not see the real share or the timing of real frames. The peer learns the real share, bounded by ρ, and the timing of real frames smeared over W; a relay learns what the wire's observer learns and nothing more.

Uniform AEAD framing across real + cover (normative). Every envelope — real publication AND synthetic cover alike — MUST use uniform **XChaCha20-Poly1305** AEAD framing. A synthetic cover envelope MUST be marked `type=cover` by the first byte of its sealed body; the `type` discriminator MUST live inside the AEAD envelope so an on-path observer cannot separate real from cover by inspecting cleartext. This XChaCha20-Poly1305 envelope/symbol framing is distinct from the streaming chunk-seal AES-256-GCM of CC 5.3.3.1 and from the HPKE-layer AES-256-GCM of the MLS Welcome wrap; they are different layers and MUST NOT be conflated.

Fragment unit — the symbol envelope. *(normative; wire-exact)* A fragment of this section is a symbol: one frame-sized piece of a real publication, or one cover frame, on a shaped link. Stored content is never split by a node: a sealed chunk travels whole, fragmented to a shaped link's frame size as above, and is replicated per item to its holders (CC 6.1.5). The realtime A/V plane (CC 5.3.3.2) emits no symbols; its chunk rides the link whole (CC 5.3.3.2.3). Each symbol is sealed independently under the `symbol_key` derivation and `symbol_envelope` layout of CC 5.4.2, diversified from the group `K_symbol` and the record's `record_id`. The per-symbol KDF is **HKDF-SHA3-256** — chosen for **hash-family separation** from the SHA-256 key-schedule hash used one layer up (a domain split between layers, so one family's weakness does not reach both), not for harvest-now-decrypt-later: the HNDL exposure of this plane is the KEM's, which is why the KEM is hybrid (CC 5.1, the hybrid X25519 + ML-KEM-768 construction); a hash choice inside a KDF has no bearing on it. Without `K_symbol` and `record_id`, a holder possesses bytes

indistinguishable from random — the structural basis for a holder's truthful cold-state-ignorance claim.

Fragment-set rebinds on MLS Add/Remove (normative). `K_symbol` and `K_record_id` are bound to the MLS group epoch (derived from the group's `ciris-scope-record-v1` exporter output, CC 5.4.1). On any MLS **Add** or **Remove** the group epoch advances, so the fragment set MUST rebind: `record_id` and `symbol_key` are recomputed under the new epoch, and subsequent symbols seal under the new keying. This carries the forward-secrecy guarantee of CC 5.1 onto the fragment layer — a removed member cannot derive the post-removal fragment set, and a newly-added member receives forward keying without retroactive symbol access except as the community's `archive_mode` permits (CC 3.2; default `rotate-forward`, 30-day window).

Reassembly (fail-honest). A receiver holding `K_symbol` + `record_id` opens each `symbol_envelope` under its recomputed `symbol_key` (XChaCha20-Poly1305 verify; a failed tag MUST discard that symbol, never accept partial plaintext); a record whose symbol set is not whole is a loss, dropped whole, never a partial decode. A chunk so reassembled is then handled as stored content: a consumer pulls the item's sealed chunks from any holder (CC 5.3.2.1 holder discovery), decrypts them with the cohort key it holds, and verifies the full digest (CC 5.3.2.5). When no holder is reachable, the fetch MUST resolve to a `ContentMiss` — an honest miss, never a silent gap or a downgrade to unverified bytes (fail-honest).

5.4.4 MLS Welcome HPKE wrap under invitee static X-Wing key (normative)

MLS Welcome messages MUST be wrapped by **HPKE `mode_base`** (RFC 9180, section 5.1.1) encrypted to the invitee's **static X-Wing KEM** public key (ML-KEM-768 + X25519; draft-connolly-cfrg-xwing-kem; confirmed in draft-ietf-hpke-pq, section 7.2). This is the admission-ceremony half of the MLS construction whose epoch rekey is specified in CC 5.1 — the same hybrid KEM (X25519 + ML-KEM-768) the MLS ciphersuite uses as its HPKE KEM.

`mode_auth` is structurally unavailable. X-Wing has no `AuthEncap` (per draft-connolly-cfrg-xwing-kem; confirmed in draft-ietf-hpke-pq, section 7.2), so HPKE `mode_auth` cannot be used. Sender authentication therefore MUST NOT be claimed at the HPKE layer; it is provided out-of-band by the **inviter's hybrid signature** (Ed25519 ‖ ML-DSA-65, as every federation-tier signature, CC 5.3.2.4.3.1) over the encapsulation bytes (below); a single-primitive signature is not admitted. Both halves of MLS ciphersuite `0x004D` are used.

HPKE parameters (cross-impl pinned; private-use suite-id, since X-Wing has no IANA KEM-id):

```
HPKE_SUITE_ID = b"HPKE-xwing-hkdf-sha256-aes256gcm-v1"
KDF           = HKDF-SHA256
AEAD          = AES-256-GCM
```

The `HPKE_SUITE_ID` string MUST be used, byte-for-byte identically on both sides, in every RFC 9180 `LabeledExtract` / `LabeledExpand` call. It is a **domain-separation byte literal, not a wire-vocabulary identifier** (CC 5.1 class rule): its hyphens are KDF input bytes and MUST NOT be snake_cased to match the `wrap_algorithm` vocabulary form. The HPKE-layer AEAD is **AES-256-GCM** — distinct from the XChaCha20-Poly1305 of the scope-privacy envelope framing (CC 5.4.3); the live A/V chunk is sealed with AES-256-GCM at every layer (CC 5.3.3.2.5).

Inviter signature. The inviter MUST sign the encapsulation bytes with the hybrid pair in the bound form every hybrid signature in this Part takes (CC 5.3.2.4.3.1): Ed25519 over the following length-delimited encoding, then ML-DSA-65 over that encoding ‖ `ed25519_sig` (no `algorithm` field; the verifier checks both halves and refuses a Welcome that carries one):

```
encap_signing_bytes
    = x25519_ephemeral_pub      (32 bytes)
    || u32_be(len(mlkem768_ct))
    || mlkem768_ciphertext
```

Signature verification is a **precondition** to unwrapping the Welcome. An implementation that omits it forfeits sender authentication, and the CC 5.2 forwarder-side membership-opacity guarantee no longer holds.

5.4.5 Witness-chain content discipline (privacy witness, normative)

A witness chain that names *what* a peer holds leaks the existence of its records as surely as a holder-discovery attestation does (CC 5.2). The content a witness chain may commit to is therefore scoped by tier, and no chain carries a count of activity: the *amount* a node carries is hidden by link shaping (CC 5.4.3), not by any chain.

> ***Distinction.*** *(normative; do NOT conflate) This is the* ***privacy witness chain*** *— a content-and-rate discipline on what a peer's witness commitments may reveal. It is NOT the CC 6.1.1* ***wholeness witness*** *(`wholeness_witness:`, the self-published divergence-detection state-root that triggers CC 5.3.2.3 quorum-merge), and it is NOT the CC 5.3.1* ***transparency-log witness*** *(an independent STH cosigner providing append-only / consistency / anti-equivocation guarantees). The three "witness" surfaces are distinct constructions, MUST NOT be cross-verified, and MUST NOT be substituted for one another.*

Federation witness chain — commits to federation-scope `record_id`s only. A federation witness chain MUST commit only to federation-scope `record_id`s: no per-community `record_id`, no count of non-federation activity, no rate counter. Where a chain pads its commitments, a cover leaf commits to `HMAC-SHA3-256(witness_signing_key, leaf_position || epoch_id)` and is indistinguishable from a real Merkle root under the standard assumptions on HMAC-SHA3-256, so an observer of the chain cannot tell which leaves are real.

Per-community witness chain — member-only visibility, signed inside the group. A per-community (CC 3.2) witness chain MUST commit to the community's `record_id` set with **member-only visibility**, and MUST be **signed inside the community's MLS encryption** (CC 5.1 epoch keying / RFC 9420 TreeKEM). The chain — like the in-group trust graph, attestation, and eviction machinery — runs *inside* the same encryption boundary that delivers outsider opacity: a non-member cannot read the community's `record_id` set off the chain, because the chain never federates beyond the group's encryption. No tier's chain carries a rate counter; what a link carries is hidden by link shaping (CC 5.4.3).

This is a normative substrate discipline, composing with the CC 5.2 structural-invisibility promise: structural invisibility keeps self/family content off the discovery surface; the witness-content discipline keeps the *witness* surface from re-leaking, at federation scope, what the discovery surface withholds.

5.4.6 Announce suppression for group-scoped destinations (normative)

A Reticulum **announce** is the broadcast that tells the whole network a destination exists and is reachable. Announcing a *group-scoped* destination therefore leaks, to an outsider who holds no envelope of the cohort, the one fact the cohort tiers exist to withhold: that the group exists at

all. The discovery discipline closes that leak — group destinations are resolved from a **cached directory + per-group HKDF**, never from an announce. This is the transport-layer realization of CC 5.2 structural invisibility on the *addressing* plane: a bounded cohort's holding claims already never leave it (CC 5.2); here the **destination itself** is suppressed at the announce plane.

No announce for group-scoped destinations (normative). A destination whose `cohort_scope` is below federation (the InvisibleEncrypted and CommunityDek tiers — `self`, `family`, `community`, `affiliations`) MUST NOT emit a Reticulum announce — **directed or broadcast; the prohibition binds the emission, not the addressing mode**. The substrate MUST suppress the announce that would reveal a group-scoped destination's existence. Only federation/Commons-scope destinations (`species`, `biosphere`, `federation`, and the `infrastructure` opt-out) MAY announce normally — they carry no anonymity claim.

The device roster is public exactly for the devices announced — per node (normative). A person's device roster is public because it has to be for them to be contactable, and it is not thereby *exposed*: it follows this section's lightnet/darknet split **one node at a time**. Each node's setup chooses whether that node **announces**, which in this section means one thing: its owner-binding (CC 3.2) is carried at `cohort_scope: federation`, owner-signed at that audience. The public roster of a person is **exactly the set of their nodes whose owner-binding is federation-scoped** — the devices they chose to be reachable on — and nothing else; the Reticulum announce of a node's transport destination, which every routing node emits, places nothing on the roster by itself. **A node that holds a `serve` seat in an `infrastructure` community MUST have an announced owner:** the point is that a named human is responsible for the node and can be found. While the node holds the seat, a live owner-binding for it MUST exist at `cohort_scope: federation`; it need not be part of the genesis and can be requested at any time. The test is evaluated live, never at a past instant: a binding made after the node was seated satisfies it, and a withdrawn binding stops satisfying it. While no such binding exists the node does not serve under the seat, refused `serve_seat_owner_unannounced`; nothing in the genesis bundle or the holders' ceremony carries or seeds the binding, which is the owner's own row. A node that boots seated and owned but unannounced announces itself if its owner's key is present and otherwise raises an alarm. The check is persist's and the server gates on it. A node whose owner-binding is held only at `self` stays on the derived plane: reachable by the person's own nodes and by whoever holds a v3 code they issued (CC 2.6.8), **never listed**, and a directory, serve policy or read route MUST NOT enumerate it to any node outside the owner's `self` cohort. This mirrors the roster discipline (a roster is never enumerable outside its cohort) at the identity plane, where re-signing the binding at federation scope *is* the opt-in, and it is the `self` cohort over the owner-binding graph that decides who reaches the unannounced remainder. **Composition with the minors floor:** a federation-scope owner-binding makes its owner contactable and discoverable by unconnected adults, which the CC 3.4.13 Q5 hard floor forbids for a minor and no steward, guardian or stacked consent may lift — so a node whose owner resolves to the `minor` band MUST NOT carry a federation-scope owner-binding, admission MUST refuse the promotion, and such a person is reachable only on the derived plane, through their own nodes and the codes they hand out.

Serve-side dependency closure (normative). A node offers an `identity:occurrence:v1` row to a peer only when it would also relay the row's attester key to that peer — the attester's owner-binding announced at `cohort_scope: federation`, by the per-node rule above; a row whose attester the receiver can never verify is withheld, not served to be refused. The test is the same when the attester is an agent identity: an agent's key carries a live steward binding (CC 3.2), and it is relayable exactly when its owner-binding is announced, by the per-node rule, no differently from any device of a person's.

Discovery is directory-cached, not announced (normative). Group members resolve each other's destinations from a locally cached directory plus a deterministic per-group derivation — no announce, no per-resolution query:

- Every participating L1 peer maintains a COMPLETE local copy of the federation directory's X-Wing public keys, refreshed via the existing `federation_keys` anti-entropy. The directory remains federation-public.
- A group destination is resolved DETERMINISTICALLY from (cached directory entry + per-group HKDF) — every member derives the same destination; outsiders, lacking the group key, cannot.
- NO per-invitation / per-resolution directory query is emitted.
- Phone-class peers that cannot hold the full directory resolve through their L1-relay parent over a relay-blinded path.

The derivation (normative). The destination is derived per member, per group epoch, from the group's MLS exporter and the member's `key_id` (the cached directory entry). Its golden vector is published with CIRISVerify:

```
S_dest        = MLS-Exporter("ciris-scope-destination-v1", context = "", L = 32)
k_destination = HKDF-SHA256-Expand(PRK = S_dest,
                  info = "ciris-edge/scope-privacy/destination/v1", L = 32)
destination   = HKDF-SHA3-256(IKM = k_destination, salt = "",
                  info = "ciris-edge/scope-privacy/destination/v1"
                         || u32_be(len(member_key_id)) || member_key_id,
                  L = 16)
```

The expand step is bare `HKDF-SHA256-Expand`, not RFC 9420 `ExpandWithLabel`. `S_dest` is a **separate** exporter from `S_record` (`"ciris-scope-record-v1"`, which feeds `K_record_id` and `K_symbol`, CC 5.4.1). Storage holds the record secret and only transport holds the destination secret, so a compromise of the record layer cannot recompute or link the addresses a node has used. Two members of one group get unrelated addresses. A destination is derived once per `(group, epoch)` at a membership or epoch change, never per packet; it rotates with install-next, activate, seal, so an epoch advance never silences a group.

What an outside observer of link traffic still sees (normative framing). Every node announces its identity destination, so an observer sees that each member node exists and is reachable. For a group it sees link requests to 16-byte derived destinations that it cannot map to a group or a member, the identity of the node that opened each link, and then sealed frames with their timing and sizes. It cannot fetch the content. What this section hides is the group's content, its membership and its address-to-group mapping. It does not hide the fact that node X opens links to some derived destination, or the shape of that traffic, which is the relationship-existence and traffic-analysis boundary of CC 5.2 and CC 1.13.3.1.

The anonymity target is outsiders, not in-group members (normative framing). Trust-enforceability to insiders and anonymity to outsiders are the same property viewed from opposite sides of the cohort encryption boundary (CC 1.13.3.4). In-group members already hold the per-group key and resolve every peer's destination from the cache — they see the group fully. The construction hides the group's existence, membership, and `querier` → `invitee` edges from **outsiders only** — an outsider who holds no envelope of the cohort — including a subpoena against the federation-public directory: because no announce is broadcast and no resolution query is emitted, the directory exposes no edge linking a resolver to the resolved destination. *Outsiders* here means the directory and any reader of wire identifiers; against an on-path forwarder's observation of link traffic the claim is CC 1.13.3.1's, not this section's. What an outsider can still

learn — that some group exists, from traffic and from federation-tier `hard_case:*` events — is the non-goal stated at CC 1.13.3.1.

Fail-secure. A peer with a stale or incomplete directory cache resolves *fewer* destinations and reports an honest miss; it MUST NOT fall back to emitting a group-scoped announce or a per-destination query to recover reachability. Per-destination announce control is a small substrate extension (informative); its absence MUST fail toward suppression, never toward announce.

A directed announce inherits the prohibition (normative). A targeted announce inherits the prohibition rather than escaping it. The purposive sentence above — suppress *the announce that would reveal* — is the prohibition's rationale, not a directed-announce exception, because on Reticulum transport no directed announce can satisfy it: multi-hop path learning **is** outsider observation — non-member transport nodes must observe the announce and retain path state for the destination, route entries a subpoena then reaches — and the epoch-bound derivation (above) forces a dilemma: a derived destination that rotates with the MLS epoch emits a synchronized roster-wide re-announce wave on every Add/Remove (cardinality, timing, membership churn — the group-existence facts this section withholds), and one that does not rotate leaves a removed member holding every peer's addressing indefinitely, breaking the CC 5.4.3 rebind discipline on the addressing plane. The fail-secure rule bans the targeted, non-broadcast per-destination *query* in the same breath: the object is the emission class, not the broadcast. What a directed announce offers is a trade of *no emission exists* (structural, honestly claimable) for *emissions exist but resist analysis* — traffic-analysis privacy, which remains a live goal and is precisely the claim base CEG/RET declines to make (CC 1.13.3.1; the Anonymous Tier is its opt-in). A reading MUST NOT spend a claimable guarantee to purchase an unclaimable one. An emission that never leaves the group's encryption reveals nothing to outsiders, and that object already exists here as the cached-directory discipline; in-group distribution of addressing material over MLS is not an announce and is not prohibited. Multi-hop reach for a group-scoped destination is therefore an **amendment-plane design question** — its bar: no outsider-observable emission, no outsider-retained path state, no epoch-correlated wave — never a reading of this clause; announce-free reach (direct link; relay-blinded resolution) is the sanctioned posture.

A scoped body MAY ride the identity-plane link when no direct path exists, and the intermediary's only role is reachability (normative). The derived address above is the zero-observer path: where a direct link exists between two members, a scoped body crosses it and nothing else. Where the only path between two members runs through a non-member transport node — a relay in the sense of CC 5.3.3.5 E1, called the forwarder below; two phones behind NATs with no member node between them — a scoped body MAY ride the members' **identity-plane link**: end-to-end encrypted between the two endpoints, the room discriminated *inside* the link encryption, and admitted by the receiver against the same address table the derived-address arrival path consults. Nothing about the derived address is emitted, resolved or retained by anyone; this rule changes what a link *carries*, never what is announced, and the prohibition above is untouched. What the forwarder learns is exactly what CC 1.13.3.1 already concedes at federation scope — that a link exists between two announced endpoints and that ciphertext flows on it at certain times and sizes — and nothing more: it cannot read the body, cannot tell a room body from a replication round, and holds no identifier that names the room. **The intermediary's sanctioned role is reachability, never custody of the conversation.** A non-member transport node exists to help two members reach each other: it answers the path request and carries link establishment. Where the endpoints can bring the link direct — a shared interface, or a direct dial on an address the path exchange revealed — the link MUST be brought direct and the forwarder drops out; where they cannot, the forwarder carries link ciphertext only, retains nothing beyond the transport table the substrate keeps for every link it forwards, and

MUST NOT be handed anything that names the room. A substrate MUST prefer the derived address whenever a direct link exists and MUST NOT route a scoped body through a forwarder when a direct path is available. What is conceded is precisely the CC 1.13.3.1 non-goal at the scope where it is already conceded: an on-path observer correlating link traffic across a clique of members can infer that a group exists, which is traffic analysis; the record layer's diversified record ids and cover framing (CC 5.4.2, CC 5.4.3) are the wire-privacy answer for the *payload* and are not a claim against that inference. The mechanism that would hide the endpoint correlation from the forwarder as well — blinded, rotating retained state, the family the Position below names — is the more secure option and is reserved for the multi-hop amendment the Position names; a deployment that needs it layers it. Minors are unaffected: reachability still requires the announce a minor's node cannot carry (the device-roster rule above).

First contact through a forwarder (normative). A peer that wants to be reachable publishes its owner binding at federation scope (the public roster above); a stranger reaches it by its federation identity, and the first message crosses a forwarder, because two peers behind NATs hold no path to each other. The forwarder's part is the one named above, reachability, and these rules keep it there. **An owned node's signed route is handed by its owner and never served by a forwarder.** The route rides inside the sealed first-contact request of the node that asks and inside the sealed acceptance of the node that answers; a forwarder serves an owned node's route to nobody, holds no contact grant, and MUST NOT be handed the consent in a form it can read. What it serves openly is what it served before: key records and identity occurrences, which are public and are what let the two peers seal to each other, and the routes of unowned nodes (itself, a bare server), which carry no person. What it holds about the pair is what CC 1.13.3.1 already concedes: that a link between two announced endpoints crossed it, or that a sealed envelope waited in a mailbox it keeps for a sleeping peer until that peer collected it, bounded in retention like the rendezvous mailbox below, ciphertext throughout. A requester therefore shows its route to the one it asks before consent; that discloses nothing the pre-seal route fetch of CC 5.3.3.5 does not already hand any dialler. Consent is the acceptance itself: a route reaches a peer only from its owner and only a peer the owner addressed, so no gate at the forwarder is needed to keep a stranger from a route, and none is permitted that would make the forwarder hold the pair. A superseding route after contact is handed the same way the first was. Whether a peer answers a request is the peer's; the receiver order of the first-contact plane (per-sender, per-link and node-wide rate) is the anti-spam, and a refused request gets no answer. **Ignore and block (normative).** The asked person MAY ignore a request: a private, local, undoable act of their own node, of which the asker is never told and after which the asker sees exactly what a pending request shows, because a visible refusal would tell everyone that one person refused another. Either party MAY block the other: a private act of their own node after which further requests from the blocked key are dropped without answer, the forwarder learning nothing, lifted only by the one who made it.

The Anonymous Tier: onion routing (normative). The Anonymous Tier (CC 1.13.3.1) hides who published a federation-scope item from the nodes that store it, from the directory, and, with shaping, from an observer of the network. The tier's packet layout, key-derivation label and AEAD are not yet pinned; until this section pins them the residual is stated at CC 1.13.3.1, and every item below is read with that status.

1. **Using it is opt-in; relaying for others is the default.** A publisher chooses to publish through an onion route. Once a node runs the tier, it relays onion traffic for others by default, because a relay set made only of volunteers would make its operators targets, and a node that runs the standard substrate is a relay like every other. A node that holds no serve seat MAY opt out of relaying by publishing the opt-out marker of CC 3.3.6.2 and

withdrawing its hop key on the same superseding row; a serve node MAY NOT: a seat carrying the marker is out of conformance as `seat_opted_out`, and a seat without a valid hop key as `seat_without_hop_key`. A node with no hop key and no marker is not yet capable of relaying and is simply not placed. **Device policy.** A phone pays for relaying in data and in a radio that never sleeps, so a phone-class node relays by default only on Wi-Fi while charging, and not on battery or on a metered connection; the user MAY override the policy either way, and the client states the cost of each setting. Servers and home nodes relay by default without qualification.

2. **First scope.** The tier carries federation-scope publication. Other scopes are not routed through it.

3. **The packet.** An onion packet is a layered packet of fixed length in which each hop's layer is encapsulated to that hop's X-Wing onion hop key (CC 3.3.6.2) and protected by a key derived from that encapsulation; each hop removes one layer and learns only the next hop. Every layer is post-quantum: a construction whose per-hop key agreement is classical alone is not conformant (CC 5.3.3.5 E1). Because ML-KEM cannot be blinded, the header carries one KEM ciphertext per hop. The packet's byte layout, key-derivation label and AEAD are pinned by this section when the tier is specified; until then the residual is stated at CC 1.13.3.1.

4. **The route and the layers.** A route has exactly three hops, one from each of three layers. **Who is a relay for a period:** a node whose current signed `transport_destination` row is live and announced, carries a valid hop key and no opt-out marker, and whose identity is owner-bound under a trust root in the genesis bundle's charter set (the bundle is the same everywhere, so every node reaches the same answer, and an unowned fresh device is never a relay); at use, a relay's layer links MUST be shaped, and an unshaped link carries no onion packet. **The deal, once per rendezvous period on the global clock:** the relays are sorted by `H(key_id ‖ period)`, serve seats first and then the others, and dealt round-robin across the three layers, seats first. Rotation by period gives a relay held back by the cap its turn. **The cap:** in each layer the other relays number at most $\lceil c \times$ conferred relays in that layer$\rceil$, with `c` a steward setting (default 2), where a conferred relay is a serve seat or an identity that holds the steward-conferred `infra:relay` role (owner-bound, rooted under a bundle root): the bound stays where trust is conferred, so no layer is ever more than two-thirds unconferred relays and a swarm of strangers cannot take a layer, and the steward grows the layers by conferring; relays past the cap wait for the next period's deal. **The floor:** a layer with fewer than three members makes the tier unavailable for that period; the tier never runs a thin layer. **Availability** is two questions. The deal answers *who*: identically on every node, from the fold. A per-node liveness measurement answers *whether now*: a relay counts as answering when it completed a shaped handshake, answered a route request, forwarded a packet the next hop acknowledged, or returned a self-addressed probe within the period; the tier is available at a node when the floor holds on the deal (else `onion_tier_unavailable_floor`) and at least three relays per layer are answering (else `onion_tier_unavailable_liveness`), and the client shows those counts, which may differ between nodes and are the honest measure of the crowd (below ten per layer the client says the set is small). Every rule here is a pure function of the fold, never of arrival order, so no coordinator assigns the layers and two nodes holding the same rows deal the same layers.

5. **The carrier.** Onion packets travel as the bodies of shaped frames (CC 5.4.3), so a link carrying onion traffic looks like any other shaped link. Each hop is an ordinary sealed, attributed link between relays: the storage node sees the last relay, never the publisher.

6. **Rendezvous.** A receiver that accepts targeted anonymous delivery publishes the route to

itself as a `rendezvous:route:v1` row (CC 3.1.4), signed by a fresh rendezvous key each hour, with the route reused for every message in that hour. The rendezvous node that holds the receiver's mailbox MUST be a serve seat, because seats do not churn and an hour-long route through a node that sleeps is lost; one seat holds each mailbox. A sender builds its own three-hop header from the directory's hop keys and appends the receiver's route.

7. **Rotation.** Hop keys, layer assignments and rendezvous indexes rotate on global clocks, never on group events.
8. **What it does not yet buy.** A relay can count the packets sent to one rendezvous in a period, because the route is reused within it. Mixing hides a publisher only among the traffic a relay carries in a window; with few users per relay the anonymity set is small, and user-facing copy says so (CC 1.13.3.4). Shaping is a prerequisite: without it the use of the tier is visible even where the path is not.
9. **Contact through the tier.** An anonymous peer has no route to hand and no address to learn: it is reached at its rendezvous route, it answers a contact request through its own circuit, and a route it holds for an announced contact is the exit of its circuit, never a destination it dials. The direct-link exchange of the contact rule above never runs over a circuit, and a forwarder offers no reflexive address on one. What the mailbox seat learns is item 8's count. A contact between an announced peer and an anonymous one therefore never goes direct, and the announced peer's client says so.

Position (informative). This design sits in the zero-emission corner of the anonymity trilemma — the membership-concealment corner, where reachability information flows only to authorized parties — with the impossibility floor ("whoever relays for you must know how to reach you") satisfied by members on the group plane; a forwarder learns only what CC 1.13.3.1 concedes. The stance it realizes is CC 1.13.3.4: anonymity at the smallest cohort scope by default, federation scope as the opt-in, claims bounded by CC 1.13.3.1. The corner has two known prices, and the design answers each. First, the **two-plane split** — the **lightnet / darknet split**: the identity plane is public (**lightnet** — announced, rooted, attributable; the federation directory announces normally and makes no anonymity claim), while the group plane (**darknet**) is *derived* from it (cached directory + per-group HKDF, above) — derivation replaces discovery, so no emission exists to observe and the classical darknet bootstrap problem dissolves. Second, **determinism replaces coordination** (CC 6.1.6): the member-relay ALM tree is a pure function every member computes identically, recovering $\lceil \log_k N \rceil$ fan-out without the coordinator that would otherwise have to learn the group exists (RFC 9750 delegates group metadata to this very layer). One corollary is recorded without minting a conformance surface: $N \to 1$ deterministic aggregation under the CC 6.1.2.1 dominance gate composes with the MLS-epoch roster, steward-binding (CC 6.1.8 N1), quorum-above-time ordering (CC 5.3 `MergeBallot`), and the CC 5.4.5 member-only witness chain into a ballot process whose *occurrence* is invisible to outsiders at scopes below federation — a classical secret ballot hides the vote; structural invisibility hides the election. For any future multi-hop amendment, the blinded-retained-state family is the nearest admissible relaxation under the bar above, with its rotation rule made explicit: rotation clocks must be global, never group-event-driven.